arXiv:2611.10141v1 [cs.CL] 6 Nov 2026

Retrofitting Unbounded Context onto Stock Language Models

Mark Oskin
Professor
School of Computer Science and Engineering
University of Washington
mhoskin@uw.edu

November 2026

Abstract

A language model that never had to forget would keep every document it has read and every turn it has spoken, and answer from any of it. This paper shows that two stock models can do that on unmodified weights by manipulating their geometry. Rotary attention turns each pair of key dimensions, a *plane*, at its own rate. The slow planes turn once in thousands of tokens and carry the address a query matches when it reads a far key. The fast planes turn once in a few tokens, and the order of the tokens inside a span is already written into the span's keys and values by the time the span leaves the near window. At distance the position blurs, and what attention reads is a bag rather than a sequence.

Each attention head learns a function. A "copy head" finds an earlier occurrence of the current token and copies from it, weighting a longer aligned match more. A head turns itself off by attending to a "sink" token. A nearby match is positional; a far match is position free. Under competition, a far span is read only if it is visible at the query of the question's own handle word. A read through a description rather than a word fails at distance: description-matched reads that hold at 95 percent at the front of the window fall to 0–16 percent at 32K and 128K in every model measured, a 70B included.

These facts allow a trick. What sits in the far part of attention does not have to be the history as it was; it can be installed ahead of the prediction. The design keeps a near window of 8K tokens at their true positions and changes what lies beyond it with the prompt: every far key and value goes to the host exact, a word index finds the units the request's rare words name, a far window of slots inside the trained range holds them while the request is encoded, and a rare word the model generates fetches its units mid-generation with one exact re-encode. The same geometry compresses the store: keeping 28 of 64 key planes at four bits loses no read on Mistral-7B, although the engine keeps the store uncompressed. Implemented in llama.cpp on Mistral-7B and Llama-3.1-8B, the design reads planted single needles at 99–100 of 100 and three-key discrimination at 88–100 of 100 from 32K to one million tokens. On RULER it scores 70.8 against 69.9 for dense attention on Llama-3.1-8B at 128K (a near window alone scores 14.8) and 67.8 against 75.5 on Mistral-7B at 32K. At 128K it prefills at 1.85 times and decodes at 1.65–1.68 times stock speed. Two-hop questions reach parity with dense attention on Llama-3.1-8B (92 against 92 at 128K) and stay 25 points below it on Mistral-7B at 32K (66 against 91), bounded by the first hop, which the near window alone loses.

1 Introduction

A language model is trained to attend over a fixed number of tokens, its context window (e.g. 32,768 tokens for Mistral-7B, 131,072 for Llama-3.1-8B). Everything the model can use about a

document must fit in that window, and a long document, a code base or a long conversation often does not. Training a model to a longer window costs more in proportion, and so does serving it, since the memory attention keeps grows with the window. We ask whether a model that was never trained beyond its window can read a history of any length anyway, with no retraining and no change to its weights. This paper shows that two stock models can, and that the reason lies in how attention already treats the distant part of its window. The idea present here is generally applicable to a wide class of language transformers.

Attention works by matching. For every token the model has read it keeps a key and a value; to produce the next token it compares a query against every key and reads the values of the keys that match best. Position enters through rotary encoding [169]: each key is turned by an angle proportional to its position, in many independent planes that turn at different rates. The fast planes turn once every few tokens and tell neighbors apart; the slow planes turn once in thousands of tokens and tell regions of the document apart. We measured two stock models, Mistral-7B [79] and Llama-3.1-8B [63], and find that beyond a near window of a few thousand tokens only the slow planes still matter. A distant token is matched by the words it carries and by its rough region rather than by its exact position, so its key can be presented at any position inside the trained window and still be found, and the order of the tokens inside a distant span was already written into the span's keys and values while the span was near. The far part of the window is therefore a bag of spans rather than a sequence, and a bag can be filled with whatever the model needs rather than with whatever came last.

What a far read needs is specific. It is lexical: a distant passage is found by a word it shares with the question, and a passage that is only described, with no word in common with the question, is not found at distance by any model we measured, a 70B model included, although descriptions do work near the question. It is bound at one moment: when several passages of the same form compete, the one the model reads is the one present when the key word of the question is encoded, so a passage fetched later is not read until the question is encoded again. And the values are the payload: a wrong value is worse than no value at all. This behavior means we need not understand the concepts behind a token to fill the far window with useful history; we need only match on the tokens themselves.

This paper describes how to do that. We split the window into two parts, near and far. The near window is the last 8,192 tokens, kept at their true positions as the model would normally see them. The far window is a fixed band of 16,384 slots inside the trained range whose contents we choose for each request. Every key and value that leaves the near window is stored on the host exactly as written; the GPU cannot hold a million tokens of them, but host memory can. We index this history by its unusual words, those whose surprisal under a running model of the document exceeds a threshold, and keep each word's last 16 occurrences with the sentence around each. When a request arrives we look up its unusual words, fill the far window with the sentences in which they last occurred, and only then encode the request, so that the passages are present when the question is read. When the model generates an unusual word while answering, we fetch that word's sentences as well, rewind to the start of the request, and encode the request and the tokens generated so far again with the sentences present; the result is identical to what a fresh pass would produce. Every setting is derived from the model's configuration. Figure 1 follows one passage through the store, the look-up and the far window. We implemented this as a small patch to llama.cpp [59].[1]

The engine reads planted needles, a seven-digit code hidden in a long document and asked for by name, at 99 to 100 of 100 from 32K to one million tokens, and when three codes of the same

[1]Our patch is released as open source at https://github.com/mark-oskin/unbounded-attention.

form are planted and one is asked for, at 88 to 100 of 100. On RULER [74], a public long-context benchmark of thirteen tasks, it scores 70.8 against 69.9 for ordinary full attention on Llama-3.1-8B at 128K, and 67.8 against 75.5 on Mistral-7B at 32K. At 128K it reads a document at 1.85 times the speed of unmodified llama.cpp and produces tokens at 1.65 to 1.68 times its speed, because every query attends over about 25K cells rather than 128K, and a million tokens run on one H200 at 8,222 tokens per second of reading and 149 of generation. The limits are specific: a question whose answer needs two hops is bounded on Mistral-7B by the first hop, which the near window alone loses, and a description alone does not match at distance, as under full attention, unless the model first names the thing in the near window. The technique generalizes beyond one document: the far window holds keys and values rather than text, so any text encoded once through the model can enter the window later, which turns retrieval-augmented generation [17, 102, 156] into retrieval into the attention rather than into the prompt.

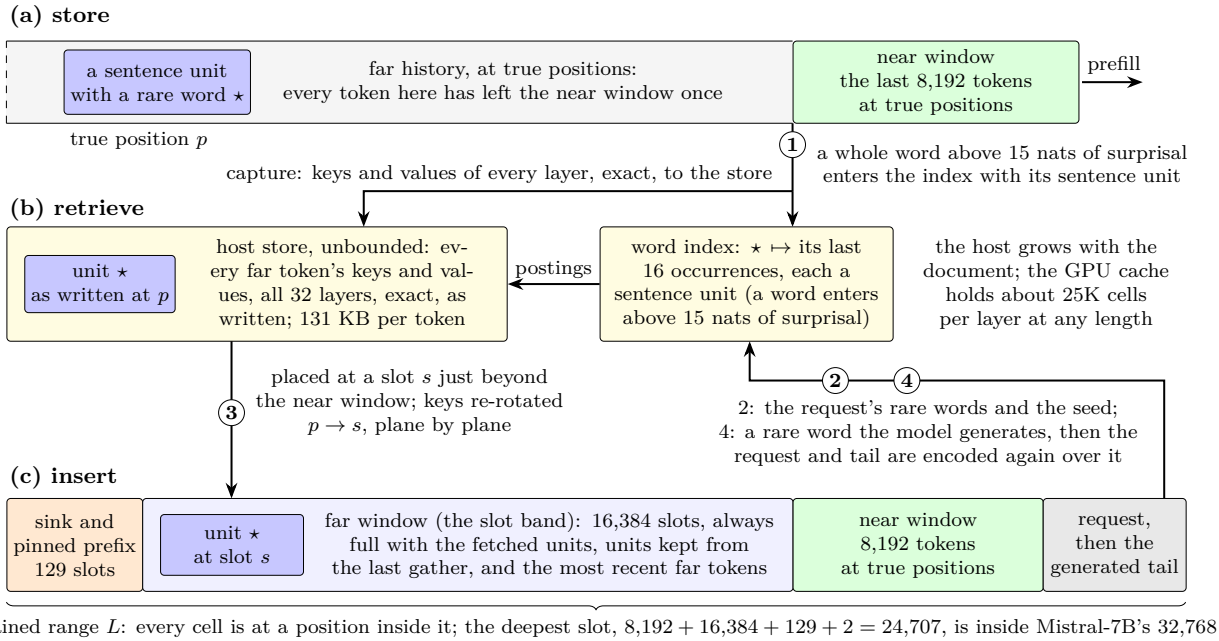

Figure 1: The life of a far unit, numbered in the order it happens. (a) Store: prefill runs the near window over the document; the keys and values of every token that leaves the window go to the host exact, as written at the token's true position, and a whole word above 15 nats of surprisal enters the word index with its sentence unit (1). The document is unbounded and the host store grows with it. (b) Retrieve: before the request is encoded, its rare words and the seed look up the index (2); the postings name the units, and the store supplies their keys and values. (c) Insert: a unit is placed at a slot s in the far window, just beyond the near window and inside the trained range, its keys re-rotated from p to s plane by plane (3); the slow planes address it at s, and the order inside it is as written. The request is then encoded over the far window. A rare word the model generates looks up the index the same way (4); the cache is then rewound to the start of the request, and the request and the generated tail are encoded again, exactly, with the fetched units present. The GPU cache holds about 25K cells per layer at any document length.

Our contributions, in the order the paper makes them, are these.

1. A census of what attention does, over every head of a 7B model reading long prose: most of its mass routes within the nearest 64 tokens, a few heads in the middle of the stack do the far reading by copying from an earlier occurrence of the current word, and the sink token is where a head puts its mass when it has nothing to say (Section 2.1).

2. How a far read works in a stock rotary model: it is matched through the slow planes and tolerates any position inside the trained range, it is bound at the moment the question's key word is encoded, it needs the model's own values, and it is lexical, with reads by description ending at the front of the window (Sections 2.2–2.6).

3. A serving design that keeps the recent window dense, stores every far key and value on the host, and fills a bounded far window from a word index before each request, with its settings derived from the model's own configuration, and its implementation as a small patch to llama.cpp (Sections 3 and 4).

4. Measurements of the implementation from 32K to one million tokens: planted needles and two-hop questions on 100 documents per cell, compared with ordinary dense attention and with a recent window alone; the 13 tasks of the RULER long-context benchmark on both models; and throughput against unmodified llama.cpp (Section 5).

5. A way to reach far context by description rather than by word, in which the model first names the described thing and the name fetches the far text, with the cost of a far window kept always full measured on it (Section 6).

Section 7 states the limitations of the measurements. Section 8 discusses how the approach can improve RAG by pre-encoding retrieved text through the model, so that it enters attention as keys and values rather than as prompt. Section 9 examines the extensive related work in this space, and we conclude in Section 10.

2 How attention behaves

In this section we explore how a stock transformer uses attention: where its attention goes, which reads carry information, and what a read needs from the key it lands on. We collect the data in this section from two stock models: Mistral-7B [79], trained to 32K, and Llama-3.1-8B [63], trained to 128K, with Llama-3.1-70B where model size is the question. We fine-tune nothing and change no weight. We measure with two code bases: a PyTorch harness with full control of the cache and its masks, and a modified llama.cpp with the cached positions editable. Several of our measurements present a far unit to the model at a fixed position just past the near window instead of at its true distance, called *placed in range* below; Section 2.3 shows that the placement changes nothing about what is read. The section opens with an accounting of what attention spends its mass on (Section 2.1) and then measures what the planes do (Section 2.2), where a far key may sit (Section 2.3), when it must be present (Section 2.4), what its value must be (Section 2.5) and which reads survive at range (Section 2.6). Section 2.7 summarizes and points the way forward for how to exploit attention's behavior and limitations.

2.1 What is attention doing?

Through training, attention heads specialize: each learns input and output projections that make it perform a function. Before changing how they work, it is worth asking what they do now. We took a census of all 1,024 heads of Mistral-7B over 100 windows of 2,048 tokens drawn from four held-out Project Gutenberg novels, 102,400 queries in all; every share below is for windows of that length. For each head and each query, we assign the head's mass to one of twelve operations by where the key sits relative to the query: the query's own token, the previous token, each of the tokens two to eight back, the bands 9–16 and 17–64 back, an induction target (the token after the

last occurrence of the current token, keyed on the token or on a mid-layer state), a copy target (an earlier occurrence of the current token, or the token before it), the matching quote or bracket, a sentence or paragraph start, a number or format token, and content tokens 65 or more back. Mass on the sinks (the beginning-of-sequence token, the window's first 64 positions, newlines, and any key taking more than 5 percent of the window's mass) is excluded from every share and counted separately as the head's rate of abstention. Mass on function tokens 65 or more back matches none of the twelve and is the residual. Table 1 gives the pooled shares, Table 2 the heads and the sinks behind them, and Figure 2 shows how the shares move with depth. "Far" in the census means 65 or more tokens back inside a 2K window; "at range" in the rest of the paper means beyond the 8K near window. The link between the two is by layer rather than by distance: the layers whose heads read far content inside the window are the layers where a planted item is read at range, as the lookup paragraph below shows; head-level overlap between the two was not measured.

Every head needs a *sink*. This is a token that is (nearly) always in the history that the head can focus on when it is not going to return a result. During the softmax, if no other result is worth carrying into the residual the sink is chosen, and its value is innocuous. Without the sink the head injects useless noise into the residual and that noise degrades the prediction accuracy. Sixty percent of the raw mass lands on the sink keys, most of all in the early layers (Figure 2, right; Table 2). The sinks have enormous residual norms [173, 195]: at layer 2 of Mistral-7B the beginning-of-sequence token's residual and the first newline's are hundreds of times the median token's, while later newlines are ordinary. The mass they hold is not a read: spreading a head's sink mass uniformly over its last 64 keys costs more than zeroing the head at the layers tested, so the sink sets the softmax floor, and a head that puts its mass there has chosen to say nothing. Besides the sink, the first few dozen tokens of the document serve as *anchors*: keys that carry no content the query needs but hold the softmax's denominator where the model expects it, and the far function tokens of Table 1 play the same role. The anchor is less specific than the sink, and the first few dozen tokens suffice.

Table 1: Where non-sink attention mass goes, pooled over all 1,024 heads of Mistral-7B on 102,400 queries from 100 windows of 2,048 tokens. Each head's mass on each query is assigned to the operation whose key it lands on; sinks are excluded from every share. The twelve named operations cover 82.7 percent of the non-sink mass. Function tokens 65 or more tokens back match none of them and hold the remaining 17.3 percent.

family	operation	share	family	operation	share
routing	self (the query's own token)	7.2%	lookup	far content, 65 or more back	**18.4%**
routing	previous token	8.1%	anchoring	far function tokens, 65 or more back	17.3%
routing	tokens 2–8 back	18.1%	induction	token after the last occurrence	1.7%
routing	tokens 9–16 back	6.4%	induction	same, keyed on a mid-layer state	1.7%
routing	tokens 17–64 back	12.7%	copy	an earlier occurrence of the token	1.5%
structure	sentence or paragraph start	2.5%	copy	the token before that occurrence	1.0%
structure	number and format tokens	3.2%	structure	the matching quote or bracket	0.1%

Routing that builds the current position's state, the query's own token, the previous eight tokens and the two local bands, is half of the non-sink mass. The far content read is under a fifth, far function tokens about the same, and induction, copy and the structural operations together about a tenth. The heads carrying that mass are mixtures rather than instances of one operation (Table 2): by the share of a head's mass on its best operation, almost every head is diffuse, and the median head's best operation holds under a quarter of its non-sink mass. Ranked by best operation, nearly half of the heads read far content first and most of the rest route locally. The mass has a long tail across heads as well: a third of the heads carry half of the non-sink mass. A head spreads its

Table 2: The heads and the sinks behind the census. The same 1,024 heads and 102,400 queries as Table 1. A head's best operation is the one that takes the largest share of its non-sink mass.

measure	result
the heads (all 1,024 of Mistral-7B)	
purity, by the share of a head's mass on its best operation	1 near-pure, 18 mixed, 1,005 diffuse (residual excluded); 1, 91 and 932 with far function tokens counted as an operation; the median head's best operation holds 23 percent of its non-sink mass
best operation, heads ranked	far content 490; the previous token 214; the band 17–64 back 208; the query's own token 69; the other six operations 43 together
the long tail of mass across heads	323 heads carry half of the non-sink mass, 619 carry 80 percent, 860 carry 95 percent
the twelve operations folded into four families	191 heads near-pure local, 159 mixing two families, 674 mixing local reading with far content and far function tokens
heads with a lift of at least 3 on a single-token aligned match (Table 4)	107; of them 101 scale with the match length by at least 1.5 times, five are flat and one falls; the duplicate alignment gives the same picture (pooled 1.72, 3.80 and 6.08 at $m = 1$, 2 and 3; the heads that fall there are layer-0 heads); all 13 shared reader heads scale
the sinks	
share of raw mass on the sink keys	0.598 over the 102,400 queries; 84 percent at layer 2, 35–39 percent at layers 0 and 13
rate of abstention per head (at least 90 percent of the mass on the sinks)	quartiles 0.4, 3.8 and 22.6 percent
residual norm of the sink tokens at layer 2, against the median token's	the beginning-of-sequence token 614 times, the first newline 712 times; the second, third and tenth newlines and the median over 502 newlines about one times
spreading a head's sink mass uniformly over its last 64 keys	costs more than zeroing the head at layers 0 and 8

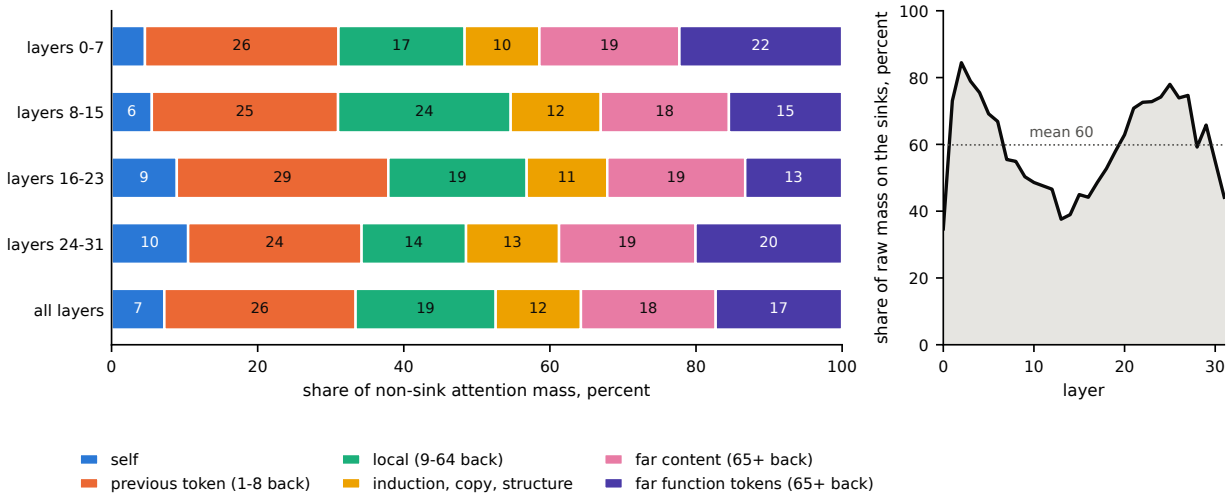

Figure 2: Left: the six families of Table 1 as shares of non-sink mass, by layer group and pooled. The mix hardly moves with depth: routing (self, previous and local) holds 48–57 percent in every group, the far content read 18–19 percent, far function tokens 13–22 percent. Right: the share of raw mass, sinks included, that lands on the sink keys at each layer, 60 percent on average, 84 percent at layer 2 and 35–39 percent at layers 0 and 13.

mass smoothly over distance, so the near-purity that head-labeling studies report [34, 181] appears here only when the local operations are merged into families. Every head runs several operations at once, and the softmax decides which one is heard.

The self, previous-token and local operations move information into the current position. They do not forecast, per se, but rather act as functions that gather information that might be used (Table 3). For each of the five biggest operations, the 16 heads carrying most of its mass were turned into a rule in two forms, "copy what followed the current token inside the attended span" and "the token after the attended span," and scored by the top-1 agreement with the model the rule adds to a fixed lexical forecaster on held-out positions, with the heads chosen on the other half of the novels. The self, previous-token and near-band operations add almost nothing in either form, and the 17–64 band adds less than the best control that uses no attention. Every operation in this family reads within 64 tokens of the query.

The far content read is the one operation whose attended span forecasts the next token beyond what a count of earlier occurrences already gives. We chose sixteen reader heads, copy heads in the plain sense that each reads from an earlier occurrence of the current token, on one half of the four novels and tested them on the other half, in both directions; 13 of the 16 are chosen on both halves, all in layers 15 to 22 (Figure 3). The rule "copy what followed the current token at the occurrence these heads attend to" fires at a tenth to a fifth of positions and adds more top-1 agreement than any rule built without attention (Table 3). Figure 4 sets it against the same copy with the occurrence chosen without attention: the latest occurrence, the occurrence with the most frequent successor, a random occurrence, and all heads summed each recover part of the gain; four text-only choosers of the occurrence land within a tenth of a point of each other at a little over half of what the heads give, and the heads still add most of their gain on top of the best of them. These are the retrieval heads of Wu et al. [191] seen from the forecasting side. The same exhaustive search over all heads finds that every one of the best-forecasting heads does so through this copy, never through the token after the span, and that they sit in the middle and late layers. A planted item read at range lands on the same layers: a working read of a far needle in the harness puts its attention on the far keys with a peak at layer 15.

The reader heads do not simply match the last few tokens before the query. A text-only rule that picks the earlier occurrence with the longest matching suffix, the same tokens before the occurrence as before the query, picks the same occurrence as the heads about half the time. Where the two differ and the heads are right, about a tenth of the cases, the occurrence the heads copy from usually shares no preceding token with the query at all, and it is usually not the most recent occurrence (Table 3). It is typically the same scene or the same referent, tens to hundreds of tokens back: "to pay the" is completed from "to tell the [coach]" in the chariot scene just before, and "said the" from "said my convict to the [sergeant]" hundreds of tokens back. The heads choose by content that a suffix match does not see. The claim has limits. The cases where the two differ are, by construction, cases of low suffix overlap, and the base rate of such occurrences among all candidates was not measured, so the heads choose beyond suffix matching rather than apart from it. The comparison also considers only earlier occurrences of the current token; that the far read is lexical at all rests on the verbatim-against-description rows at the end of this section and in Section 2.6. Section 2.3 shows that the read is position-free as well.

Induction, keyed on the token or on a mid-layer state, copy of an earlier occurrence and of its predecessor, and the structural operations (quotes and brackets, sentence and paragraph starts, number and format tokens) hold about a tenth of the non-sink mass between them. Feucht et al. [51] separate, by ablation, a token route that copies verbatim from a concept route that carries meaning across paraphrase and translation. The census is consistent with that account from the outside, as a behavioral dissociation: the token route is a count, which token followed the current

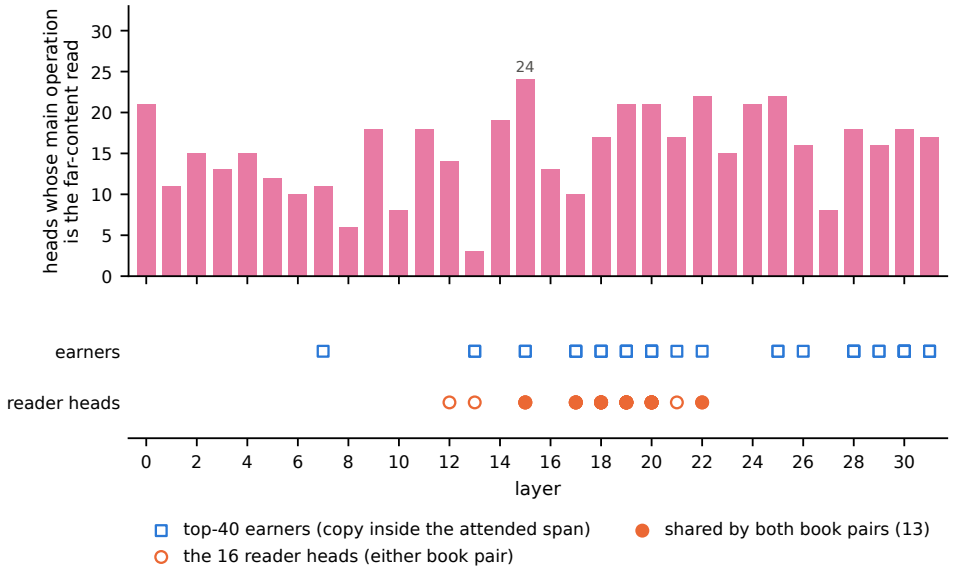

Figure 3: Where the far readers live. Top: the number of heads in each layer whose best operation is the far content read, 490 of 1,024 in all; every layer has some, 145 of them in layers 15–22. Bottom: the 16 reader heads chosen on either half of the novels (open circles) and the 13 chosen on both (filled), the union in layers 12–22 and the shared 13 in layers 15–22; and the 40 heads whose attended span forecasts the next token best (squares), every one of which forecasts by copying what followed the current token inside its span.

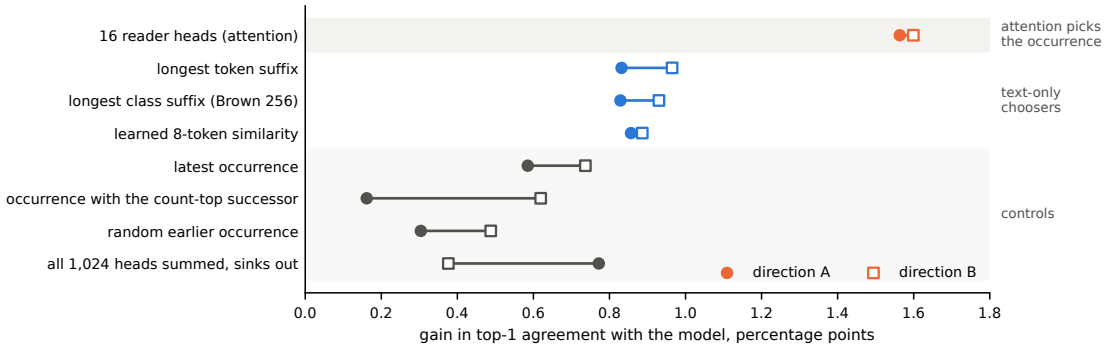

Figure 4: What the reader heads' choice of occurrence is worth. Every row copies what followed the current token at one earlier occurrence of it; the rows differ only in who chooses the occurrence. Gain is the top-1 agreement with the model added to a fixed lexical forecaster on the held-out half of the novels (direction A: 237,568 positions; direction B: 230,633), choices and thresholds fixed on the other half. The 16 reader heads add +1.56 / +1.60 points; the best text-only chooser +0.96; the latest occurrence +0.59 / +0.74.

Table 3: What each operation forecasts. Every rule copies from the span the named heads attend to, or copies at an occurrence chosen without attention; the gain is top-1 agreement with the model added to a fixed lexical forecaster on the held-out half of the novels, in both directions, with heads and thresholds chosen on the other half.

rule	form	result (direction A / B)
rules built from the 16 heads carrying most of each operation, scored on about 470,000 held-out positions		
self, the previous token, 9–16 back	copy what followed the current token inside the span; the token after the span	at most +0.08 points in either form
17–64 back	the same two forms	+0.29 / +0.27 against +0.77 / +0.74 for the best control that uses no attention; "the token after the span" at precision about 0.01 for every local operation
far function tokens, 65 or more back	the 16 heads carrying it most (layer-0 heads L0H25, L0H23, L0H22 and L0H19 lead)	+0.14 / +0.06, below every no-attention control
far content: the 16 reader heads	copy what followed the current token at the occurrence the heads attend to	fires at 20 / 10 percent of positions, precision 0.47 / 0.62, +1.56 / +1.60 points
the same copy with the occurrence chosen without attention (Figure 4)		
the latest occurrence		+0.59 / +0.74
the occurrence with the most frequent successor		+0.16 / +0.62
a random occurrence		+0.30 / +0.49
all 1,024 heads summed, sinks removed		+0.77 / +0.38
four text-only choosers: the longest matching token suffix, the longest suffix over 256 word classes, a learned similarity over the previous eight tokens, a recency tie-break		+0.83 to +0.96, within 0.1 point of each other, 53–60 percent of what the heads give; the heads add +0.90 / +0.94 on top of the best of them
where the heads are right and the longest-suffix chooser is wrong		
share of the rule's firings		9.7 / 10.8 percent (4,531 / 2,614 cases)
preceding tokens the chosen occurrence shares with the query		none in 78 / 80 percent of cases, at most one in 91 / 92 percent
the chosen occurrence is the latest one		40 / 35 percent; over all firings the heads pick the chooser's own occurrence 44 / 51 percent of the time
distance of the chosen occurrence	10th / 50th / 90th percentile	13 / 72 / 695 and 17 / 127 / 870 tokens
the best-forecasting heads, exhaustive over all 1,024, and the read at range		
the 40 heads whose attended span forecasts best		every one through this copy, never through the token after the span; layers 7 to 31, 37 of the 40 in layers 15–22 and 25–31
a working read of a far needle at 128K (harness, $n = 16$)	attention on the far keys	mean 0.067, peak 0.235 at layer 15

token before and how often, and a count of earlier successors built from the window alone reproduces it, which is why the latest occurrence with no attention recovers a good part of the reader heads' gain and why the four text-only choosers cannot be told apart. What the reader heads add is a choice a count cannot make. No head was ablated here, so the identification stays behavioral. The classic induction head [46, 142] is the token route seen one head at a time; at under two percent of the mass it is a small part of what attention does on prose.

The induction route is longer than one token. For every query x and earlier key p in the census windows, let m be the length of the aligned match that ends at the key's predecessor, the number of consecutive $k \geq 0$ with $\mathrm{tok}(x-k) = \mathrm{tok}(p-1-k)$, so that $m = 1$ is the classic induction target and $m = 3$ a key that follows a three-token repeat of the query's own recent past. Table 4 gives the attention on such keys as a lift over the head's mean attention at the same distance, with the sinks excluded. Pooled over all heads and weighted by mass, a key that follows a two-token match

draws more than twice the attention of one that follows a single-token match, and a three-token match more again. Nearly every head with a strong single-token lift scales with the match length, and the duplicate alignment ($\mathrm{tok}(x-k) = \mathrm{tok}(p-k)$) gives the same picture (Table 2). All 13 shared reader heads scale, some by an order of magnitude from a one-token match to a two-token match. D'Angelo et al. [37] find the same exponential weighting of a context match by its overlap in induction circuits trained on synthetic data, and the generalized induction head of Kim et al. [91] adds fuzzy matching to the count; the census shows the same rule at work on a 7B model reading prose. The reader heads therefore do two things that the chooser analysis above keeps apart. When the query's recent past repeats somewhere earlier, they weight the longer repeat more, which is induction over n-grams rather than over one token, and when no such repeat exists, the case in four of five of their disagreements with the suffix chooser, they still choose an occurrence, by content the suffix does not see.

Table 4: Attention against the length m of the aligned match, Mistral-7B on the census windows (100 windows of 2,048 tokens, 192 million query-key pairs after the sink exclusions). Lift is the attention on a key over the head's mean attention at the same distance bin, so 1 is no preference; restricting to keys 65 or more tokens back changes no entry by more than a point. The pooled row weights every head by its mass; the 13 rows are the shared reader heads of Figure 3.

	induction alignment, lift at $m =$			duplicate alignment, lift at $m =$			mass on $m \geq 1$
head	1	2	≥ 3	1	2	≥ 3	ind. / dup.
all 1,024 heads, mass-weighted	1.71	3.80	4.6	1.72	3.80	5.6	
L15H21	2.1	7.9	7.5	16.3	100.0	124.3	0.03 / 0.29
L17H26	5.1	19.8	19.0	1.8	5.5	5.7	0.08 / 0.02
L17H27	1.9	6.1	6.4	5.5	33.0	40.9	0.03 / 0.09
L18H0	20.7	176.3	147.0	0.6	5.6	5.9	0.51 / 0.01
L18H2	15.9	79.7	86.1	0.7	3.3	4.2	0.31 / 0.01
L18H11	2.2	4.9	4.9	1.6	4.4	6.0	0.03 / 0.02
L19H8	5.3	20.5	22.7	1.4	3.6	4.4	0.09 / 0.02
L19H17	4.8	11.7	12.0	1.1	3.5	4.1	0.06 / 0.02
L19H19	2.0	5.9	7.1	1.7	4.9	6.0	0.03 / 0.02
L20H6	7.0	52.3	66.7	1.1	4.9	5.9	0.16 / 0.02
L20H20	2.0	4.3	4.2	3.6	9.2	12.7	0.03 / 0.05
L20H23	1.9	4.3	3.9	11.1	47.4	64.1	0.02 / 0.18
L22H29	3.2	7.8	8.9	0.9	2.0	2.7	0.04 / 0.01

Function tokens 65 or more tokens back, commas, articles and newlines far from the query, hold about a sixth of the non-sink mass but match no operation. The heads that carry most of it are layer-0 heads, and as a rule through the 16 heads that carry it most, that mass adds less than every no-attention control (Table 3). It behaves as the sink does: mass parked on keys that carry no content the query needs, holding the softmax's denominator where the model expects it. An anchor the model cannot read is worse than none: outside the trained range the sink itself stops holding the floor (Section 2.3). In other words, the grammatical scaffolding a query uses comes from within 64 tokens; far function tokens hold mass and carry nothing the query reads.

We measured everything above on prose. A planted item tells the same story at range (Table 12). A question that names the item's handle word verbatim is answered about nine times in ten 32K back on Mistral-7B and 128K back on Llama-3.1-70B. A question that describes the same item with no shared content word is answered nine times in ten or more when the item is a thousand tokens back and almost never 32K and 128K back in every model measured, each with three same-format decoys present and full attention over the whole document (Section 2.6). In other words, a question about "the pet" does not reach a far passage about "the cat", though a cat is a pet; only the word "cat" does. What reaches far back in the attention window is the exact word rather than

its description.

2.2 The planes

Rotary attention [169] groups dimensions into pairs. Each pair of key dimensions is a *plane* that turns with position at its own rate: on Mistral-7B, with a rotary base of 10^6 and 64 planes per head, the fastest plane completes a turn every six tokens and the slowest every five million. A key may be viewed as 64 clock hands. The slow hands say which neighborhood of the document the key belongs to; the fast hands say which seat inside it.

A query and a key agree on a plane only while the plane's turn between them is under about a radian, so the number of planes a key is coherent on falls with its distance, from most of them a few hundred tokens back to a quarter at the edge of the 32K trained range (Table 5). The count follows from the rotary schedule alone. The slow planes serve as the index card: they carry the match between a query and a far key, and they carry it at any distance. The fast planes serve as the page: they turn once in a few tokens, and whatever order a span's tokens carry is already in the span's keys and values by the time the span leaves the window. The measurements below separate the two jobs in time: the slow planes do their addressing when a query reads, and the order inside a span is settled earlier, while the model writes the span's keys.

The 15 slowest of Mistral-7B's 64 planes carry most of the score between a query and a far key, two and a half times what they would carry if the planes were interchangeable, and the share is the same at every distance. Position-sensitive heads, those whose far scores change under a small jitter of the key positions, concentrate in layer 1, and more than a third of the heads are flat under the jitter. On 8K documents under dense attention at true positions, keeping only the 15 slowest planes of every key older than 1,024 tokens costs a few hundredths of a nat, less when the cut starts later or keeps more planes, and planted needles read in full under it; keeping the 15 *fastest* planes instead costs several nats and reads nothing, because it destroys the sink (Table 5). The far match lives in the slow planes, which is the observation behind analytic plane cuts for far keys [10, 67, 69], the finding of Chiang and Yogatama [31] that long-distance retrieval rides on the lowest-frequency dimensions, and the key-channel pruning family [185, 200, 207, 221].

The fast planes carry signal out to a few hundred tokens, and that signal is consumed while the span is near: the cut above costs less the later it starts, so what the fast planes contribute is concentrated in the near past. Who reads them there is visible in the weights: most key-carrier reader heads have their strongest upstream writer in layers 0–1, and those writers attend one token back far more than the head population does, previous-token heads found from the weights alone. What a far key then carries of its neighborhood is an ordered bag: the contributions of the tokens at small offsets enter the key at falling norms, and a question about the order of two far mentions is answered well above chance with every far position collapsed onto one. The order of the tokens inside a span is written into the span's own keys and values while the span is inside the window; the weights name the local previous-token heads of Section 2.1 as its writers, by inference from the writer-reader graph rather than by a write-time test.

Every far cell, the needle's seven digits included, placed on one identical position reads every single needle (Table 5). A far key cut to its 22 slowest planes still reads three-key discrimination in full (Table 6). And rotating the fastest planes of every far key by the token's offset inside its own span rather than by the key's position reads every single needle and every three-key discrimination at the same loss as rotation by position. The one thing a far span cannot do without is the distinctness its tokens were written with: far keys synthesized from one key per word, so that every digit of a code shares one key, are addressed correctly and read nothing. Slow planes address at read; order is settled at write.

Table 5: What the planes carry. Mistral-7B unless stated; the plane-cut rows are 8K documents under dense attention at true positions, the agreement rows the harness at 32K with $n = 16$ except where the table says llama.cpp.

measurement	result (Mistral-7B unless stated)
the slow planes address	
planes a key is coherent on, by distance (the plane's turn under about a radian)	39 of 64 at 200 tokens back, 26 at 3,000, 21 at 10,000, 18 at 20,000 and 15 at the edge of the 32K trained range; from the rotary schedule alone
share of the query-key score carried by the 15 slowest planes	59 percent, against the 23 percent that 15 of 64 would carry if the planes were interchangeable; the same at every distance (one 8K document, far keys 1K to 8K back)
heads flat under a ±64-token jitter of the far key positions	391 of 1,024; the position-sensitive heads concentrate in layer 1
keeping only the 15 slowest planes of every key older than 1,024 tokens (8K documents, dense attention at true positions)	+0.032 nats ($[+0.014, +0.051]$ over seven documents); +0.007 when the cut starts at 4,096; +0.006 and +0.0007 for 24 and 32 planes past 1,024; +0.26 for 8; the model-card rule that keeps every plane coherent at 1,024 tokens (31 planes) +0.0014; planted needles 38/38 under the 15-plane cut against 100/100 unpatched
keeping the 15 fastest planes instead	+6.85 nats and 0/30: the sink's share of mass falls from 0.459 to 0.0005
the fast planes write the order	
who writes the fast-plane signal (from the weights)	of 248 key-carrier reader heads, 167 have their strongest upstream writer in layers 0–1; those writers' measured one-back attention is 0.49 against 0.018 for the head population
what a far key carries of its neighborhood	the tokens at offsets 0–4 enter the key at norms 4.9, 2.6, 1.8, 1.5 and 1.3
the order of two far mentions, every far position collapsed onto one	answered at 0.750 $[0.598, 0.858]$, $n = 40$
order is settled at write	
every far cell, the needle's seven digits included, placed on one identical position	7500/7500 single needles (Table 7)
a far key cut to its 22 slowest planes	three-key discrimination 16/16 (Table 6)
the 21 or 27 fastest planes of every far key rotated by the token's offset inside its own span rather than by position (32K, harness, $n = 16$)	16/16 single and 16/16 three-key; loss 2.1940 / 2.1943 and 2.1938 / 2.1939 against 2.1942 for rotation by position
far keys synthesized from one key per word, so that every digit of a code shares one key	addressed correctly (0.7–0.99 of the far mass lands on the unit) and read at 0/16

Table 6 keeps the k slowest planes of every far key and drops the rest, at four-bit keys (groups of 32, per-head centering) and four-bit values, in the harness on Mistral-7B at 32K with $n = 16$ documents, each holding one seven-digit needle and two same-format competitors. Three-key discrimination holds in full from 64 planes down to 22 and loses two reads at 15; single needles read in full at every rung; loss is identical to four decimals at every rung. A far key cut to its 22 slowest planes still reads; cut to 15 it does not, and the cliff between the two is shallow. The 22 planes are exactly those whose wavelength is at least $2\pi \times 8{,}192$ tokens, every plane still coherent across a span the length of the near window. Bit width is free at every rung: four-bit keys and values read the same single and three-key rows as eight-bit and as bf16. Loss does not resolve the cliff: across the whole ladder it moves by less than a thousandth of a nat while the three-key read is what moves. The ladder is measured on Mistral-7B; Llama-3.1-8B's rungs are unmeasured under this protocol.

2.3 The far match is position-free

The far match does not use the key's position (Table 7). On Mistral-7B with read-time edits of the cached positions, single planted needles read in full under 25 different edits, including every far key moved to one position and the needle moved to the far edge of the trained range; moved

Table 6: The plane ladder. Mistral-7B at 32K in the harness, $n = 16$ documents, one seven-digit needle and two same-format competitors per document, the far unit placed in range and visible from the first token of the question; keys kept in their k slowest planes at four bits (groups of 32, per-head centering), values at four bits. Bytes are the counted size of one token's keys and values at each rung; the 64-plane row is the full bf16 key and value.

planes kept (of 64)	bytes per token	single needle	three-key	loss, nats
64 (bf16, uncut)	131 KB	16/16	16/16	2.1944–2.1948
40	33.0 KB	16/16	16/16	2.1944–2.1948
32	30.0 KB	16/16	16/16	2.1944–2.1948
28	29.0 KB	16/16	16/16	2.1944–2.1948
22 (every plane coherent across 8,192 tokens)	27.5 KB	16/16	16/16	2.1944–2.1948
15	24.8 KB	16/16	14/16	2.1944–2.1948

beyond the range the read degrades, a little at twice the range and badly at three times it. The one placement that fails is distance zero, the current token's own phase. Presenting every far key at one position just beyond the near window, regardless of where it really was, is therefore free: the whole far history placed that way reproduces full attention at 32K on loss and on needles, where the same keys scored at their true graded distance lose half the needles, and a set of far units placed together at one distance reads nearly all. Position-free storage with re-rotation at a new position is the primitive of CachedAttention [54], ReAttention [114], MiniPIC [143], LazyAttention [193] and CacheFocus [100]; what is specific here is the measurement that the far match tolerates any placement inside the range, degrades beyond about twice it and loses its anchors outside it, and that the keys placed this way are read in the same layers as the reader heads of Section 2.1.

Moving a key is likewise free. A key rotated at one position and then re-rotated to another by the difference is the same key as one rotated once at the new position: a single shift of the whole sequence by up to a million positions changes the top-1 token at almost no decoded position, and the dense logits at that offset are the same to five decimals (Table 7). Inside the range, where a key sits is a number the model does not check; Section 4.1 checks the two ways an engine can make the move.

Outside the trained range the anchors fail, the sink first. A sliding window whose sink sits at its true distance loses more than half a nat on Mistral-7B and more than a nat on Llama-3.1-8B by one million tokens; with the sink placed in range the same window is flat to one million. The first 128 tokens of the document, kept as anchors at their true distance, cost several nats at 1M. Inside the trained range a key may sit anywhere; beyond it the match degrades with distance and the anchors the softmax stands on are lost.

2.4 Residency and binding

When must a far unit be present? With a same-format competitor in the document, the answer is a moment rather than a span: the unit must be visible to one particular query. Table 8 places the needle's unit in range with no retrieval step, so that the unit is simply there, and masks it so that only queries from a chosen position onward can see it, in two harnesses, with and without three same-format competitors in the document.

The binding forms at the handle's own query. With three competitors, a unit visible from the handle word is read in full; visible from one, two or four tokens before the first digit, with the handle's query excluded, it is read about half the time; visible only at the predicting query, a quarter of the time. Without competitors the requirement relaxes but does not vanish: nearly every read survives a few tokens of lead, and half survive visibility at the predicting query alone. The

Table 7: Where a far key may sit. Read-time edits of the cached positions, placement of the whole far history, shifts of the whole sequence, and the anchors outside the trained range.

measurement	result
single planted needles under read-time edits of the cached positions (Mistral-7B, llama.cpp, $n = 300$)	7500/7500 under 25 different edits, including every far key moved to one position and the needle moved to 30K back; moved to 40K and 60K on the 32K-trained model 0.993 and 0.960; at 100K 0.213 (0.696 per digit); distance zero, the current token's own phase, is the one placement that fails
the whole far history placed at one position just beyond the near window (Mistral-7B at 32K, $n = 16$)	loss 2.1863 against 2.1857 under full attention; needles 14/16 against 14/16; the same keys scored at their true graded distance 2.1937 and 7/16
a set of far units placed together at one distance of 8,192 tokens	23/24
a single shift of the whole sequence by 2^{17} to 2^{20} positions	the top-1 token changes at no more than one of 256 decoded positions; key cosine at least 0.9992
dense logits at a rotary offset of 2^{20} against offset 0	differ by 6×10^{-6}
a sliding window whose sink sits at its true distance, to one million tokens	+0.58 nats on Mistral-7B and +1.15 on Llama-3.1-8B; with the sink placed in range the same window is flat (2.198 ± 0.002 and 2.4512 ± 0.0005 nats over six lengths from 32K)
the first 128 document tokens kept as anchors at their true distance, at 1M	+3.9 to +5 nats

Table 8: When the far unit must be visible. The unit is placed in range and masked to queries from the stated position onward; the question names the handle word and asks for the seven-digit code. The needle harness is the harness of the planted-needle rows above; the description harness is the one of Section 2.6. With competitors present the binding forms at the handle word's own query: one position earlier reads, one position later reads nothing. The last group is the chain that no prefill regime binds.

the unit is visible from	no competitors	three same-format competitors
Mistral-7B at 32K, needle harness: $n = 16$ without competitors, $n = 8$ with		
the handle word's own query	16/16	8/8
one / two / four tokens before the first digit, handle excluded	14/16 / 14/16 / 15/16	3/8 / 3/8 / 4/8
the query that predicts the first digit	8/16	2/8
6 / 20 / 256 tokens before the handle	16/16 / 16/16 / 16/16	
the window's tail only, evicted at the question	0/16	
throughout, evicted before the digits	0/16	0/8
Mistral-7B at 32K, description harness: $n = 28$ without competitors, $n = 81$ with		
one position before the handle	25/28	75/81
one position after the handle	0/28	0/81
the first token of the request / after the handle word, many units at once ($n = 56$)		53/56 / 0/56
Llama-3.1-8B at 128K, description harness: $n = 8$		
one position before / after the handle		7/8 / 0/8
a chain reached only through a referring unit, the question sharing no rare word with the answer unit ($n = 16$)		
encoded with full attention / the near window alone / only its last window, read at range	0/16 in every case, as the unreachable control	
the same chain inside the window	16/16, four reads above a 12/16 format-copy floor	

description harness draws the same line: with three competitors a unit visible from one position before the handle is read nearly always on both models, and from one position after it never, with and without competitors. Nothing earlier than the handle adds anything: visibility from 6, 20 or 256 tokens before it reads in full each time, and residency in the window's tail alone, evicted at the question, reads nothing. The unit must also stay: evicted before the digits, the read is lost in the harness and in llama.cpp alike, with the generated name surviving in the residual stream and the verbatim copy lost. The line holds when many far units become visible together: a set of units visible from the first token of the request reads nearly all, and the same set visible only after the handle word reads none. This is the residency side of the query-aware contextualization result of Liu et al. [112], at fixed order and matched mass: the same content, the same position, only the moment of visibility varied. It is also the mechanism behind the query-visibility audits of Luo et al. [123] and the wait-to-attend memory of Attendre [206], which keeps evicted queries so that a later key can still be attended by them.

The binding also does not form at prefill time across units. In a chain whose question shares no rare word with the answer unit and reaches it only through a referring unit, a document encoded with full attention, with the near window alone, or with only its last window reads nothing at range in every case, as does the unreachable control, against a full read for the same chain inside the window (Table 8). The referring unit's cached keys and values carry nothing of the answer unit under any prefill regime. A second hop is work the model does while it generates.

The same rule governs a unit that becomes visible while the model is generating (Table 9). In llama.cpp, with no unit supplied at the question and the needle's unit made visible only once the model has generated its handle word, so that it first affects a prediction two tokens after the word closes, a plain copy question reads the late arrival nearly always on both models with no further step. With one same-format worked example in the request, neither model reads it at either length, with the unit present and unmasked. Encoding the request and the generated tail again with the unit present, which the engine does by rewinding the cache to the start of the request and rolling forward, recovers most of the reads on Mistral-7B and all of them on Llama-3.1-8B; the pass is identical in its logits to a fresh prefill on both models, and the design calls it the re-encode. It also takes the chained hop on Mistral-7B from about half to all. The description harness draws the same picture on the description side: a unit that arrives at the model's own naming of it sits among the far keys, unmasked in all 32 layers, at a small fraction of the attention a resident unit receives, and the verbatim read goes from almost none to all once the question is re-encoded over it. Presence is not residency: under competition the binding forms when the handle's query runs, and a unit that was absent then stays unread until that query runs again; without a competitor the late arrival is read as it stands.

2.5 Values

Keys decide what is fetched; values decide what is read. Table 10 collects what a far value must be. Values are as word-clustered as keys: two values of the same word far apart are more alike than two context neighbors at every depth, by a wide margin in the early and late layers, and a ridge fit on word identity beats a fit on the neighbor mean at every one of the 32 layers. The asymmetry between keys and values is in the consequence of getting them wrong. Far keys reduced to the token's identity alone cost almost nothing on loss, and read no needles; far values reduced the same way cost a large fraction of a nat. Replacing every far value in an 8K document with the token's own bare identity code costs about half of what zeroing them costs; a map fitted on anchor types costs more; permuting the identities costs several nats, worse than zeroing. The readers consume the per-token code: far values reduced to the token's own earlier value keep most of the

Table 9: A unit that arrives mid-generation, with a same-format worked example in the request, is read reliably only after the request and the generated tail are re-encoded over it. llama.cpp, with no unit supplied at the question: the needle's unit becomes visible only once the model has generated its handle word; $n = 16$ per cell, one run; a second run of the worked-example cells read 12/16, 15/16, 15/16 and 16/16. The chained hop, in which the second name is given, $n = 7$ on Mistral-7B and $n = 5$ on Llama-3.1-8B. The re-encode is identical in its logits to a fresh prefill on both models.

		Mistral-7B		Llama-3.1-8B	
question	re-encode	32K	128K	32K	128K
plain copy, no worked example	no	16/16	15/16	16/16	
plain copy, no worked example	yes	16/16	16/16	16/16	
copy, one worked example in the request	no	0/16	0/16	0/16	0/16
copy, one worked example in the request	yes	12/16	13/16	16/16	16/16
chained hop, second name given	no / yes	4/7 / 7/7		5/5 / 5/5	

description harness, the unit arriving at the model's own naming of it: unmasked in all 32 layers, it draws 0.05–0.08 of the attention a resident unit receives; the verbatim read goes from 1/81 to 81/81 once the question is re-encoded over it

read's worth, the context remainder alone keeps a fifth, its share rising to about a third in the last layers, and a wrong code costs seven times deletion; the copy-head reader at layer 20 is the most remainder-driven reader of all, so the mid-stack readers of Section 2.1 lean on the remainder more than the population does. In other words, a wrong far value is worse than no value at all.

Table 10: What a far value must be. Mistral-7B unless stated; the replacement rows are one 8K document with the keys left real, the read rows $n = 16$ documents.

measurement	setting	result
same-word cosine of values, against context-neighbor cosine	layers 2 / 16 / 31; pairs over 64 tokens apart; chance ≤ 0.01	0.77 / 0.31 / 0.28 against 0.01 / 0.26 / 0.11
loss with far values replaced, keys real	one 8K document; real loss 1.919 nats	zeroed +0.82; token-identity code +0.44; map fitted on anchor types +0.71; identities permuted +2.8 to +3.1
what the readers consume	far values reduced to the token's own earlier value	84% of the read's worth kept (+0.044 against +0.266 for deletion); context remainder alone 18%; wrong code +2.0 nats, seven times deletion
synthesized keys, one vector per token	from one shared layer-8 vector through a per-layer map: cosine 0.85 per word type, 0.98 per occurrence, a shortlist within one to six points of the real keys; per-token keys at cosine 0.94	needle read 0/16 on both models
synthesized values	keys real; addressing intact (0.7–0.99 of the far mass on the unit)	read 0/16; Llama-3.1-8B loss: synthesized keys +0.386, synthesized values +0.017
values rebuilt by a forward over the unit's own tokens	no preceding context; key cosine 0.84	16/16 at 32K, 16/16 at 128K, 14/14 at 1M; bulk loss +0.018 nats over all 776 eligible units of a 32K document, +0.0003 with 256 tokens of preceding context

No key or value synthesized outside the model reads. Every head's key for a word is predictable from one shared layer-8 vector through a per-layer map at high cosine per word type and higher per occurrence, which ranks a shortlist within a few points of the real keys and is dead for serving attention: per-token synthesized keys read no needle on either model, and synthesized values leave

the addressing intact and destroy the readout, at a small loss cost against a large one for synthesized keys. A forward pass over the unit's own tokens with no preceding context, at a lower key cosine than the map reaches, reads every needle from 32K to 1M: a self-consistent forward pass outperforms a higher-fidelity map. Its bulk cost over every eligible unit of a 32K document is a few hundredths of a nat with no preceding context and almost nothing with a few hundred tokens of it. Only the model's own values read, whether written in place or rebuilt by the model from the text; the asymmetry agrees with the cache-side finding that values tolerate less approximation than keys [35] and with the accuracy cost reported for predicted keys and values [73]. Multi-token words are assembled into single vectors (Table 11). A multi-token word is assembled in the residual at its last piece between layers 2 and 6 on both models: the embedding table's rows predict human relatedness for pairs of multi-token words far worse than for single-token pairs, and the layer-5 to layer-6 residual lifts the multi-token pairs to the single-token level while the single-token pairs barely gain. The assembled vector is order-coded, it is built by the first feed-forward layers, and read through the embedding table it names a single-token partner far better than the logit lens does. We speculate that this inability to synthesize keys and values arises from the frame the model chooses during training to operate in [145]. Synthetic values have to be generated in the right frame, otherwise they appear as noise to the model. Future work may find a way to generate keys and values outside the model by working in that frame.

Table 11: A multi-token word is assembled as a token. Both models; relatedness is Spearman correlation with human ratings over 6,607 rated word pairs.

measurement	result (Mistral-7B / Llama-3.1-8B)
human relatedness predicted from the embedding table's rows, over 6,607 rated word pairs	Spearman 0.15 / 0.06 for pairs of multi-token words, against 0.63 / 0.62 for single-token pairs
the same from the layer-5 to layer-6 residual at the word's last piece	multi-token pairs 0.59 / 0.51; single-token pairs gain at most 0.06 over their table rows
the assembled vector is order-coded	cosine to the intact vector 0.21 with the pieces reversed and 0.27 shuffled, against a 0.96 single-token ceiling
built by the feed-forward layers 1–3	ablating them takes the best multi-token head's discrimination from 0.92 to 0.62
read through the embedding table	names a single-token partner at a median rank of 7 of 13,571, where the logit lens ranks it 96

2.6 Far reads must be precise

Figure 5 and Table 12 plant a seven-digit code with a handle word and ask for it by description, with no content word shared between question and item, three same-format decoys present, a one-shot worked example in the document tail, and full attention over the whole document, all in the PyTorch harness. At the front of the window the description is resolved: nine times in ten or more a thousand tokens back on every model, falling with distance on the 7–8B models and staying above three quarters to 20K on the 70B. At range the read is gone in every model, the 70B included, with and without the worked example. The verbatim handle survives the same trip on every model. Scale closes the in-window gap and the dependence on a worked example: on the control haystack at 3K the 70B reads descriptions far better than the 8B, with and without a worked example. It does not close the far read. The collapse is a property of the read rather than of model size, and it is the in-cache form of the collapse NoLiMa measures when the needle shares no word with the question [134].

Table 12: Reads at range by description and by word. Description-matched questions with three same-format decoys, full attention, PyTorch harness, $n = 81$ per point unless stated ($n = 82$ for the 70B at range). Mistral-7B on a Project Gutenberg haystack; the two Llama models on a control haystack inside the window and, at range, the Gutenberg haystack (8B) and the control haystack (70B).

	Mistral-7B 32K	Llama-3.1-8B 128K	Llama-3.1-70B 128K
the description, inside the window (percent read; worked example in the document tail)			
1K / 3K / 5K back	95.1 / 71.6 / 63.0	88.9 / 60.5 / 44.4	98.8 / 97.5 / 84.0
10K / 20K back		28.4 / 17.3	97.5 / 75.3
3K back with no worked example (control haystack)		8.6	81.5
at range			
the description, with / without the worked example	6/81 / 9/81	7/81 / 13/81	0/82
the verbatim handle	72/81	15/16	77/82
placement and content matching do not rescue the description read			
the needle's unit placed in range, three competitors ($n = 25$): the verbatim handle / the description	24/25 / 1/25, against 20/25 / 1/25 under full attention		
far units ranked by a per-head content vector: the described unit placed in range / read		27/27 / 0/27	
the word, once the model has produced it (full attention, harness)			
a scratchpad that first names the described thing	9/81 to 56/81	13/81 to 61/81	
given a correct name		59/59	

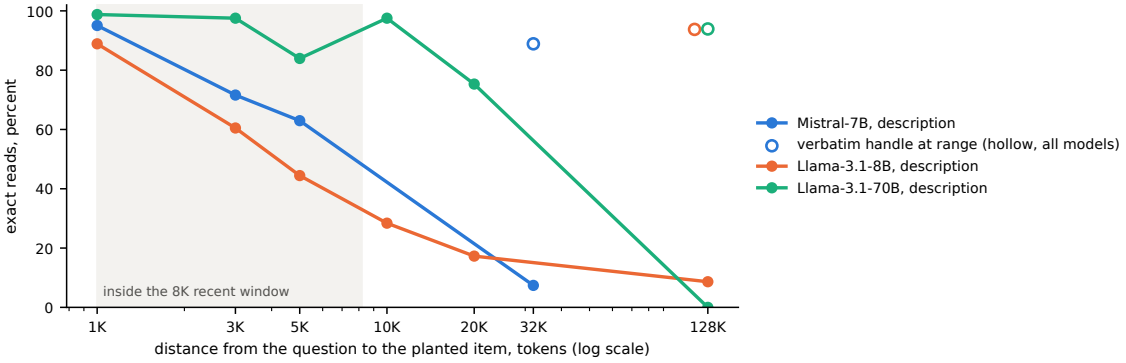

Figure 5: Description-matched reads against distance under full attention, three same-format decoys, worked example in the document tail, $n = 81$ per point ($n = 82$ for the 70B at 128K). Mistral-7B on a Project Gutenberg haystack; the two Llama models on a control haystack to 20K and, for the far points, the Gutenberg haystack (8B) and the control haystack (70B). Hollow markers are the verbatim-handle question on the same items at range (the 8B marker is offset left for visibility). Every model resolves the description at the front of the window and none at range; the word crosses the window in every model.

Placement does not rescue it (Table 12). On Mistral-7B at 32K with three competitors, the needle's unit placed in range reads the verbatim handle better than full attention over the document does, and reads the description almost never under both: a nearer position with fewer competitors is a better place for the lexical read than the far document itself, and it does nothing for the description read. Putting the right unit in front of the model by a content match does not rescue it either: far units ranked by a per-head content vector put the description's unit in range every time on Llama-3.1-8B at 128K and the model never read it. The description route is a property of the model rather than of where the key sits, and it lives at the front of the window at every scale. What does cross the window is the word, once the model has produced it: under full attention in the same harness, a scratchpad that first names the described thing lifts both models from about one in seven to about seven in ten, and given a correct name Llama-3.1-8B reads every code. Section 6 turns the name into a retrieval step.

2.7 Summary

Each of the facts above constrains how a long history can be served. Routing, half of the mass, reads within 64 tokens of the query and is the one place where every plane must be right, so the recent past has to be present at its true positions. The one operation that forecasts is a few mid-stack heads copying an occurrence chosen by the words it contains, the more sharply the longer the aligned match, so a far span is worth having back for its words, and a description of it is no address until the model has turned the description into a word. The far match rides the slow planes and tolerates any position inside the trained range, degrading beyond about twice it, so a far span can be brought back at whatever position is convenient, while the sink and every anchor must sit inside the range. The order of a span was written into its keys and values while it was in the window, and the readers copy values, so a span must come back as the model wrote it; a synthesized value destroys the read. Under competition the binding forms at the handle's own query, so a far span must be visible before the question is encoded and must stay while the answer is copied, and a span that arrives later stays unread until the question is encoded again over it. Section 3 is those constraints turned into a design, every setting of which the model card fixes.

3 Design

Section 2 describes how attention operates at the geometric level. In this section we use that information to retrofit unbounded context onto an already trained transformer. The overall design is described in Figure 1. A request's life has three phases. Prefill runs window-only and streams the keys and values that roll off the near window to the host while the index is built. The hand-off gathers for the request's words and the seed, places the units at slots, and then encodes the request over them. Decode fetches on every generated rare word, rewinds the cache to the start of the request, and encodes the request and the tokens generated so far again over what arrived.

Each of the design's parts has a purpose (Section 2.1). Routing is served by a near window of dense attention at true positions. The far read is lexical, so the index is a word index: a unit is fetched for the rare words it contains. The far read is position-free, so a fetched unit is placed at a slot inside the trained range, at whatever distance is convenient, and its slow planes carry the match there; that is the fact the whole retrieval tier is built on: a far unit is worth fetching for the words it contains, and once fetched, its precise position is not relevant. The reader heads copy an occurrence and what followed it nearby, so a unit's tokens must arrive in the order the model wrote them, along with that context. The copy is a read of a value, so values are stored, never synthesized (Section 2.5). Attention relies on a sink token to turn a head off and on the first tokens

of the document as anchors for its softmax floor, so we keep the sink and the first 128 document tokens inside the trained range at all times (Section 2.3). Nothing in the design is per head or per layer; the same far window serves every layer alike.

3.1 Parameters

Table 13 lists every knob of the design. None needs the user's intervention: the defaults are derived from the model card of the transformer. Models must be trained with per-plane rotation, so we do not support multi-head latent attention [38] or a linear-bias scheme [151], and the design's rule refuses a configuration whose deepest slot would lie outside the trained range, a check the engine does not perform (Section 4). The one exception to derivation is the plane count, which we set by measuring Mistral-7B and finding how large a safety margin is needed; the engine does not apply it. The measurements behind the rows are those of Section 2: the plane count in Section 2.2, the range rule and the sink rule in Section 2.3, the event rule in Section 2.4, and the bit widths in Section 2.5.

3.2 Dynamically constructed window

The near window ($W = 8{,}192$ tokens) of the attention window is stored densely and in the order the transformer would normally see it. The sink and the pinned prefix of the first 128 document tokens are kept for the anchoring mass of Section 2.1, and kept at slots inside the trained range. What goes into the remainder of the window, the far window of $S = 16{,}384$ slots, we vary dynamically. Every cell the model sees sits at a position inside the trained range, because a key presented beyond the range is unreadable (Section 2.3): a fetched key is presented just beyond the near window regardless of where it really was, and inside the range that placement is free. Table 14 holds the measurements behind this section.

The host store. Every token that leaves the near window keeps its real keys and values on the host, 131 KB per token at f16 across the 32 layers. The store is unbounded, and nothing is evicted from it: attempts to limit it did not work, because it is not possible to know what a later request will ask for. The host side of the design is the frame of the offloaded retrieval systems as well [27, 98, 109, 194]; what differs here is what finds a unit in the store and where the unit is placed.

Compression. The geometry of attention points at an obvious compression of the host store. At distance not all planes are readable; the fast ones appear as noise to the model. Analytically only 22 planes are meaningful for Mistral-7B (Section 2.2), and we retain 28 to provide a buffer. Keeping those planes at four bits shrinks the store per token to about a fifth of its uncompressed size, 131 KB to 29 KB. We measured the cut in the harness on Mistral-7B and validated it on three-key discrimination rather than on loss, since loss barely moves across the whole ladder while the three-key read is what moves. The engine stores its keys and values uncut, and Llama-3.1-8B's rungs are not measured.

The word index. The store is indexed by word rather than by token, because the far read is lexical. Words are whole words at the tokenizer's piece boundaries; a word enters the index only if it is uncommon, above 15 nats of summed surprisal on a running unigram model of the document, and the index keeps a word's last 16 occurrences. The two rules bound the demand of a gather by the query rather than by the document: at one million tokens a gather asks for about a hundredth

Table 13: Every knob of the design, derived from the model card, with the measurement behind it. W is the near window, S the slot band, L the trained range. Values are the configuration on Mistral-7B and Llama-3.1-8B; we measured the two compression rows in the harness, and the engine stores the host store uncompressed. In the engine W is a lower bound: prefill above 24,706 tokens runs in 1,024-token batches, so a query sees W to $W + 1{,}023$ recent tokens.

knob	value (Mistral-7B / Llama-3.1-8B)	rule, from the model card	the measurement behind it
trained range L	32,768 / 131,072	the hard ceiling: refuse if $W + S +$ prefix $+ 2 > L$	a key presented outside the range is unreadable: the sink at its true distance costs +0.58 / +1.15 nats at 1M, at a slot the loss is flat to 1M (Section 2.3)
near window W, slot band S	8,192 / 16,384	split the context budget $W : S = 1 : 2$; with one given derive the other and re-check against L	the measured setting; halving the band at 128K costs single needles (0.875 against 1.000, $n = 16$)
pinned prefix	128	the first 128 tokens of the document, at slots	the measured setting; at true distance the prefix costs 3.9–5 nats at 1M
key planes kept	28 of 64 (harness, Mistral-7B; the engine stores keys uncut; Llama-3.1-8B rungs unmeasured)	the analytic cut keeps every plane coherent across W ($\lambda \geq 2\pi W$ at the effective per-plane rate, 22 planes on Mistral-7B); the default adds one measured rung of margin, a calibration rather than a derivation	three-key discrimination against planes kept (Section 2.2, harness, $n = 16$): 16/16 down to 22 planes, 14/16 at 15; the analytic cut $\lambda \geq 2\pi W$ keeps 22, the default 28 keeps one rung of margin
key and value bits	4 / 4, groups of 32 (measured; the engine stores f16)	the smallest width that is free	bits are free: four-bit keys and values read the same single and three-key rows as eight-bit and bf16, loss within 0.0025 nats across the sweep
key centering	per head	the per-head mean over the first W tokens, then frozen	measured free in the harness; W is the span the window holds anyway
sinks	BOS; on Mistral-7B the first newline	BOS always; any other token whose layer-2 residual norm is over 50 times the running median	BOS is 614 times and the first newline 712 times the median token at layer 2; the second, third and tenth newlines and the median of 502 newlines are about 1 times
admission	15 nats	a word enters the index only above 15 nats of surprisal on the running unigram model; a valve raises the threshold when demand exceeds $S/2$	demand at 1M falls from 200,002 to 1,700–2,800 tokens per gather, coverage from 0.08 to about 1.0; the needle unit ranks first among the hits; 32K unchanged
occurrences kept	16 per word	keep the last 16, oldest dropped	bounds demand by the query, words $\times$ 16 $\times$ unit length, about 8,000 tokens at any length
units	sentences	whole words from the tokenizer's piece boundaries; sentence units for every source	sentence units win or tie every needle cell at far less band traffic (about 700 tokens per request against 9–14K for window units with the far seed off; 1,062–1,768 at 32K and 128K, 2,342–3,058 at 1M, in the rows of Section 5)
events	15 nats, budget 2,048	the same threshold on generated whole words; one exact re-encode of the request and generated tail per event, within a per-turn token budget	a unit arriving mid-generation is read only after the re-encode (Section 2.4); the gate loses no read
sliding-window layers	window only	a layer that cannot see past its own window gets no band	by construction
out of class	refuse	MLA, and non-rotary schemes such as ALiBi	slot placement re-rotates keys plane by plane; neither has the per-plane rotation that makes a slot meaningful [38, 151]

Table 14: The measurements behind the design. Reads are exact matches of n; harness rows are the PyTorch harness on Mistral-7B at $n = 16$, and the rest are the llama.cpp engine. Section 2 holds the measurements of attention the rows rest on, and Section 5 the results.

part	what we measured	result
the host store and its index		
store size	keys and values of every far token at f16, 32 layers	131 KB per token; 128.5 GiB at one million tokens on Mistral-7B
admission at 15 nats, last 16 occurrences	tokens a gather asks for at 1M, and coverage of the planted item	200,002 tokens per gather without the two rules, 1,700–2,800 with them; coverage 0.08 to about 1.0, the item's unit ranking first among the hits; nothing changes at 32K
sentence units against ±256 / ±512 windows	needle cells, band traffic, bulk loss, RULER volume tasks	sentence units win or tie every needle cell (two-hop on Llama-3.1-8B at 128K 84 against 66–68) at about a tenth of the band traffic; windows win bulk loss by 0.009 nats on code and 0.001–0.004 on prose, and the volume subset of RULER on Mistral-7B, 55.7 against 47.6
whole words against fixed chunk boundaries	single needles, harness	14/16 when the chunk cuts the handle, 16/16 with whole words
compression of the key planes (harness, Mistral-7B)	three-key discrimination against planes kept; bytes per token	16/16 down to 22 planes, 14/16 at 15; the analytic cut (wavelength at least $2\pi W$) keeps 22, the default 28 keeps one rung of margin; 29.0 KB per token at 28 planes and four bits, against 33.0 at 40, 27.5 at 22, 24.8 at 15, and 131 KB uncompressed; loss moves by less than 0.0005 nats across the ladder
a store budget	usage-at-exit eviction of whole units on a 1M document, capped at 512K and 256K tokens	single needles 8/16 and 5/16, three-key 2/16 and 2/16, against 16/16 unbounded; every evicted unit had never been used
the far window		
slot placement	fetched keys at a slot just beyond the near window, against their true distance	free inside the trained range (Section 2.3); at its true distance the sink costs +0.58 / +1.15 nats at 1M and the prefix 3.9–5 nats, at slots the loss is flat to 1M; fetched keys scored at their true distance beyond 128K read worse than the near window alone at every length
fill per request	tokens fetched, of 16,384 slots	1,062–1,768 at 32K and 128K, 2,342–3,058 at 1M; the back-fill covers about two thirds of the far region at 32K, 16K of 123K far tokens at 128K, one in sixty at 1M
hand-off before the request is encoded	the identical gather, visible from the start of the request or only after the handle word	53/56 against 0/56 (78/81 at $n = 81$; 52/55 with a 300-token worked example in the request); in llama.cpp at 128K 16/16 against 13/16 single needles
keep-fill: topping up from the far window's own contents	single needles; the ring's advance per gather	at 32K 0.812 to 1.000 on both models, at 128K 0.875 and 0.938 to 1.000; the advance 2,396 to 1,046 cells
the far seed	two-hop; the handle being seeded; three-key	+3–13 points; the handle seeded in 97–100 of 100 requests, from 76–82; three-key within 3
the far window through generation	a unit evicted before the digits are produced	0/16, in the harness and in llama.cpp alike
the always-full far window	the description read through naming on Mistral-7B (Section 6)	36/81 on the engine against 51/81 with only the fetched units resident; denying the band the same-format decoys recovers 9 of the 15 points
events and prefill		
an event's re-encode	tokens covered, time on one H200, exactness	30–80 tokens, 31–37 ms; byte-identical in its logits to a fresh prefill; the unit first affects a prediction two tokens after the word closes
the surprisal gate at 15 nats, the per-turn budget	reads in the worked-example cells ($n = 16$); re-encodes per 128 decoded tokens	12/16, 13/16, 16/16, 16/16, no read lost; 0 / 0 / 4 on Mistral-7B and 0 / 1 / 3 on Llama-3.1-8B at 8K / 32K / 128K, from three to five without the gate; budget 2,048 tokens per turn
window-only prefill	loss against streamed prefill; speed against full-depth prefill	−0.0001 nats on prose and −0.00007 on 1.1M tokens of code with cross-file references; full-depth prefill is 20 percent slower at 128K and 42 percent at 1M

of the tokens it would ask for without them, and the planted item's unit ranks first among the hits. The unit around each occurrence is its sentence, so that a handle is never cut at a chunk boundary; sentence units read every needle cell at about a tenth of the band traffic of fixed windows, which win only on bulk loss and on the volume tasks of RULER, and a window of ± 256 or ± 512 tokens around each occurrence is a user flag for code and volume-heavy use.

What to put in the far window. At the hand-off we scan the request for words that are in the index, and add the seed: the 32 rarest words of the near window's last 4,096 tokens that have an occurrence beyond it, at half weight. The seed fetches the far units of rare words the near window already mentions, which is the first hop of a two-hop question, and it lifts two-hop reads without costing three-key discrimination. A unit is scored by the summed surprisal of the distinct query words it matches, and units are taken in score order, ties by recency. The gather happens before the request is encoded, because a far unit is read only if it is visible at the query of the question's own handle word (Section 2.4): the identical gather reads nearly every needle when its units are visible from the start of the request and none when they arrive after the handle word. The far window is always full. It is filled in three layers: the units fetched for this request; then whatever it already held, newest first; then a back-fill of the most recent far tokens down from the near window's edge. Slots are ranked by recency, newer nearer the near window. At every length the fetched units are a small part of the far window, so most of it is back-fill.

Events. When the model generates a rare word, its postings are fetched and written into the far window; the unit first affects a prediction two tokens after the word closes. Retrieval that fires on what the model is generating is the active-retrieval idea [82, 170], here inside the model's own cache. A unit that arrives mid-generation is not read until the request and the generated tail are encoded again with it present (Section 2.4), so after each fetch the engine rewinds the cache to the start of the request and encodes the request and the tokens generated so far again, with the fetched units in the far window, and then continues generating. This re-encode is exact, byte-identical in its logits to a fresh prefill, and a few tens of milliseconds long. The event is gated on the generated word's surprisal at the same 15 nats as admission, which loses no read and holds the re-encodes to a few per hundred generated tokens, and it carries a per-turn budget, since tokens can regain attention late in a long reasoning trace [216].

Prefill. Prefill runs window-only. Second hops are generation-time work, and no binding forms at prefill time across units under any prefill regime (Section 2.4); gathers during prefill add nothing measurable to loss, and full-depth prefill is slower by a fifth at 128K and by two fifths at 1M. Above 24,706 tokens prefill runs in 1,024-token batches with the near window rolled per batch, so a query sees between W and $W + 1{,}023$ recent tokens, and W is a lower bound.

Eviction. The far window evicts at every gather, in the reverse of the order it is filled: the back-fill of recent far tokens goes first, then the oldest of what the window held, and the units fetched for a request stay through its generation, because a unit evicted before the digits are produced is never read. Topping up from the window's own contents before back-filling keeps a unit fetched once from being the first thing overwritten, and it is what takes single needles to a perfect read at 32K and 128K on both models.

4 Implementation

The engine's share of the work is small, because the model does the reading; the engine moves cells and maintains the store and the index. We implemented the design in llama.cpp [59] as one small change to the library and a driver that holds every policy. The library gains an entry point that moves a cache cell to a new position, re-rotating its key by the difference, together with read and write accessors for a cell's key and value and an accessor for the rotary base; nothing else in the library changes, and with the design off the binary is the stock engine. The driver captures the keys and values that roll off the window and streams them to the host, builds the word index as the document is prefilled, gathers and places units at the hand-off and at events, and re-encodes the request and the generated tail over a late arrival. Every knob of Table 13 is derived from the model card at startup and printed. The models are the public Q8_0 files of Mistral-7B and Llama-3.1-8B, the latter converted from the original weights with its rotary frequency table; f16 weights read the same strings as Q8_0 on 15 of 16 examples.

We test the engine against itself, and Table 15 holds the checks and the costs. Below the trained range the engine is byte-identical to the stock engine. Every cell of the far window and the near window carries its provenance, and the provenance checks find no bad cell. The pinned prefix, re-placed from the host at every move, stays at its true keys through the moves of a 1M prefill, where a prefix moved in place drifts. The re-encode at an event is byte-identical in its logits to a fresh prefill on both models. The performance path, which feeds prefill in batches of 1,024 tokens, captures the previous block to the host while the next computes, and captures decode per block, is byte-identical to the one-block-at-a-time path on nearly every regression text and changes no exact-match count. Two tests are not implemented: refusing a configuration whose deepest slot lies outside the trained range, and resuming a session's store. The costs are accounted per token and per request in the same table. Two parts of the implementation were measured separately: the placement path and the sink over a million tokens.

4.1 Placement by re-rotation

llama.cpp stores keys already rotated at their position and re-rotates them by a shift when they move; a pre-rotary store keeps keys unrotated and rotates them once at placement. The two are equivalent, as Section 2.3 measures and Table 15 checks on the engine: on a fixed binary the two paths produce identical strings at 1M and at 128K; on both models at 128K, single needle and three-key discrimination read in full under either capture path, with identical per-example digit shares; and a shift of the whole sequence by up to a million positions leaves the decoded tokens and the dense logits where they were (Table 7). The engine re-rotates: a pre-rotary store would cost a quarter of prefill for the same reads, and compression does not need it, since a plane cut commutes with rotation.

4.2 Keeping the sink exact over a million tokens

The engine re-places the pinned prefix from the host at every move, one exact rotation each, because a prefix moved in place accumulates rounding error. Window-only prefill moves the sink and the pinned prefix up one block at a time, and each move is an f32 rotation rounded to f16, thousands of times by one million tokens. In the slowest planes a 128-position turn is under half an f16 unit in the last place, so an in-place prefix stops turning in those planes; the angle error grows linearly in the number of moves and the cosine loss quadratically, and by one million tokens an in-place sink key has drifted far from its true value (Table 15). Every token encoded late in such a prefill is

Table 15: The engine checked against itself, and its costs. The sink rows are the diagnosis of Section 4.2.

check	setting	result
against the stock engine		
logits below the trained range	9,320, 16,448 and 24,320 tokens	0 of 32,768 logits differ
provenance of every cell	band and window cells after moves	0 bad of 786,432 band cells and 393,216 window cells
the pinned prefix after the moves of a 1M prefill	re-placed from the host at every move / moved in place	cosine 0.999999 / 0.992304 to its true keys
the re-encode at an event	both models	byte-identical in its logits to a fresh prefill
the batched performance path	64 regression texts, against one block at a time	byte-identical on 62 of 64; no exact-match count changes
placement by re-rotation		
re-rotated capture against pre-rotary capture	a fixed binary, 16 examples at 1M and at 128K	identical strings on all 16
single needle and three-key under either capture path	both models at 128K, $n = 16$ per cell	16/16 in all eight cells, identical per-example digit shares
the pre-rotary capture path	prefill cost	25 percent of prefill for no accuracy; off
the sink over a million tokens (Mistral-7B, window-only prefill)		
moves of the sink and pinned prefix	one f32 rotation rounded to f16 per block	about 8,100 moves by one million tokens
a 128-position turn in the slowest planes	against half an f16 unit in the last place	1.6×10^{-4} radians against 4.9×10^{-4}: those planes stop turning
$1 - \cos$ between the sink key and its true value	at 128K / at 1M	mean 0.0023 / 0.19 ($\times$80 for $\times$8 shifts); cosine 0.432 in all 16 examples at 1M
the fix's cost	prefill at 128K on one H200; decode	4,574 against 4,772 tokens per second (4 percent); decode unchanged
costs per token and per request		
host memory	f16	131 KB per token
GPU cache per layer	window, far window, pinned prefix	W to $W + 1{,}023$ window cells, 16,384 far-window cells, 129 pinned cells
the hand-off	32K and 128K	about 0.2 s
re-encodes	per 128 generated tokens	0 to 4 (Table 20)

computed against a drifted floor, and on Mistral-7B no single needle is read at one million tokens with the needle resident. Re-placement keeps the sink at its true value and reads every needle (Table 16), at a cost of a few percent of prefill at 128K and nothing on decode. Llama-3.1-8B reads every needle at 1M without it.

Table 16: The sink over a million tokens, the same 16 documents throughout, one seven-digit needle each. We measure the sink key's cosine to its true value before the request. The rows are the prefix moved in place, rewritten once before the question, rewritten by a hand-off before the question, and re-placed from the host at every move.

model	hand-off	pinned prefix during prefill	sink key cosine to its true value	reads
Mistral-7B	after the question	moved in place	0.432 (all 16 examples)	0/16
Mistral-7B	after the question	moved in place, rewritten once before the question	0.99997 at the question	7/16
Mistral-7B	before the question	moved in place	0.432, then rewritten by the hand-off	9/16
Mistral-7B	after the question	re-placed from the host at every move	0.99997	12/16
Mistral-7B	before the question	re-placed from the host at every move	0.99997	16/16
Llama-3.1-8B	before the question	moved in place	not measured	16/16

5 Results

We measure the implementation in llama.cpp on stock model files, on one H200, with the question placed at the end of a long document. Three planted tasks and one public benchmark carry the numbers. A *single needle* is a seven-digit code introduced by a handle word ("the secret code for the ... is"), planted far back and asked for by its handle, the arrival test of the needle protocol [88]. *Three-key discrimination* plants three same-format codes and asks for one by its handle: the discrimination test, the one that competition breaks. A *two-hop* question names an item whose far unit names the code, with the bridge sentence inside the window: the answer needs a first hop (produce the bridge word) and then the far read; we score the bridge word as well as the code. Every planted row has $n = 100$ documents from one item seed and we read it with a Wilson 95 percent interval; within one seed, a difference inside the intervals is parity, never a gain. Two references bound each row: dense attention (stock llama.cpp with the whole cache resident, where it fits) and the window-only floor (the near window and the pinned prefix with no retrieval). We run RULER [74] from its official generator, 13 tasks at $n = 100$ each, scored by its own string-match metric, and measure throughput against stock llama-bench in the same job. A second floor, run where the comparison needs it, holds a 24,576-token window with no retrieval: the design's whole GPU budget of window and far window, spent on recency.

5.1 Single needles and three-key discrimination

Table 17 is the arrival and discrimination matrix. Single needles read at or near every one of the hundred at every length on both models, at parity with dense attention where dense attention fits. Three-key discrimination reads in the high eighties to the mid nineties, and its differences from dense attention sit inside or at the edge of the intervals, so they are read as parity. Dense Mistral-7B at 128K reads nothing, because 128K is four times its trained range, so the design's reads there are reads dense attention cannot make rather than gains over it. The window-only floor reads a fifth to a third of the needles at 32K, where some fall inside the window, and almost none at 128K; the matched floor, the design's whole GPU budget spent on recency, reads about one in ten, so the read comes from the far window rather than from the size of the budget. At one million tokens nothing is lost from 128K, with a few thousand tokens fetched per request. The PyTorch harness agrees in every one of its cells from 32K to 1M on both models, where a sliding window with its sink kept reads no single needle at any length.

5.2 Two hops

Table 18 is the two-hop matrix. Llama-3.1-8B reaches parity with dense attention at 128K and sits seven points below it at 32K. Mistral-7B sits well below dense attention at 32K and reads a little over half at 128K, where dense attention is out of range. The bridge-word column locates the loss. On Mistral-7B at 32K the bridge word is produced about as often with the needle resident as by the window-only floor, and far more often under dense attention: the near window alone loses most of the first hop, the far window costs a little more, and the code follows the word. At 128K the far window's share of the loss grows, and the matched floor produces the word often and the code never. Llama-3.1-8B names the bridge word less often than its own floor does, yet answers nearly as often as dense attention: it reaches the far code without producing the bridge word, so the first-hop reading bounds Mistral-7B's result and not Llama-3.1-8B's. Retrieval is not the bound in either case: the surprisal gate never fired, since the handles sit far above the admission threshold, and no needle was cut.

Table 17: Single needles and three-key discrimination, exact matches of 100 with Wilson 95 percent intervals, one item seed. Dense is stock llama.cpp with the whole cache; the floor is the near window and pinned prefix with no retrieval; the matched floor is a 24,576-token window, the design's whole GPU budget, with no retrieval, run at 128K. †Dense Mistral-7B at 128K is four times its trained range. ‡A dense 1M cache does not fit one H200.

task	model	context	design	fetched tokens	dense	floor	matched floor
single needle	Mistral-7B	32K	99 [95–100]	1,142	99 [95–100]	18 [12–27]	
single needle	Mistral-7B	128K	100 [96–100]	1,768	0 [0–4]†	0 [0–4]	11 [6–19]
single needle	Llama-3.1-8B	32K	99 [95–100]	1,062	100 [96–100]	18 [12–27]	
single needle	Llama-3.1-8B	128K	100 [96–100]	1,604	100 [96–100]	0 [0–4]	11 [6–19]
three-key	Mistral-7B	32K	88 [80–93]	1,132	93 [86–97]	31 [23–41]	
three-key	Mistral-7B	128K	93 [86–97]	1,759	0 [0–4]†	1 [0–5]	9 [5–16]
three-key	Llama-3.1-8B	32K	96 [90–98]	1,090	98 [93–99]	31 [23–41]	
three-key	Llama-3.1-8B	128K	94 [88–97]	1,632	87 [79–92]	1 [0–5]	8 [4–15]
single needle	Mistral-7B	1M‡	100 [96–100]	3,058	does not fit		
single needle	Llama-3.1-8B	1M‡	100 [96–100]	2,342	does not fit		
three-key	Mistral-7B	1M‡	99 [95–100]	2,757	does not fit		
three-key	Llama-3.1-8B	1M‡	100 [96–100]	2,353	does not fit		

Table 18: Two-hop questions of 100: the code produced, and the bridge word produced, for the design, dense attention, the window-only floor and, at 128K, the matched floor of a 24,576-token window with no retrieval. †Dense Mistral-7B at 128K is out of range.

model	context	design code / word	dense code / word	floor code / word	matched floor code / word	fetched tokens per request
Mistral-7B	32K	66 / 73	91 / 95	0 / 75		1,080
Mistral-7B	128K	59 / 59	0 / 0†	0 / 78	0 / 84	1,759
Llama-3.1-8B	32K	87 / 46	94 / 85	0 / 64		1,104
Llama-3.1-8B	128K	92 / 52	92 / 89	0 / 72	0 / 68	1,610

5.3 RULER

Table 19 holds the 13 RULER tasks on Llama-3.1-8B at 128K and Mistral-7B at 32K. On Llama-3.1-8B the design's aggregate is a point above dense attention, far above the window-only floor and well above the matched floor, the design's whole GPU budget spent on recency. The single-needle tasks are at or near perfect, except the third on Mistral-7B, where both arms fall to the seventies. The multi-key tasks, the stress tasks, read above dense attention on the first two and a few points below it on the third, where the design misses the usual bar of 50 by three points [74]; multi-value and multi-query read above dense attention by about ten points. The design loses the tasks that chain through content, variable tracking and the first question-answering task, the same limit as the first hop above. On Mistral-7B at 32K the aggregate is about eight points below dense attention, and the largest losses are the second and third multi-key tasks, retrieval under competition, and the two volume tasks, common and frequent words, where the window-unit flag recovers part of the loss (Table 14). The multi-key losses are the far-window competition of Section 3.2 on a public task: a key with three same-format competitors is the one read the always-full far window hurts. The remaining controls the RULER bar asks for, a random store and the published training-free systems, are not in this table.

Table 19: RULER, 13 tasks at $n = 100$ from the official generator, RULER's string-match metric ×100. Dense and floor as above; *the matched floor is a 24,576-token window, the design's whole GPU budget of window and far window, with no retrieval.

	Llama-3.1-8B at 128K					Mistral-7B at 32K			
task	design	dense	floor	matched*	design − dense	design	dense	floor	design − dense
single needle 1	100.0	100.0	2.0	18.0	+0.0	100.0	100.0	22.0	+0.0
single needle 2	98.0	100.0	8.0	19.0	−2.0	100.0	100.0	28.0	+0.0
single needle 3	100.0	90.0	5.0	15.0	+10.0	77.0	72.0	21.0	+5.0
multi-key 1	99.0	87.0	8.0	18.0	+12.0	96.0	93.0	28.0	+3.0
multi-key 2	71.0	59.0	5.0	19.0	+12.0	58.0	76.0	22.0	−18.0
multi-key 3	47.0	53.0	3.0	9.0	−6.0	53.0	73.0	17.0	−20.0
multi-value	91.3	78.5	8.0	16.8	+12.8	94.8	99.2	23.2	−4.5
multi-query	96.5	86.2	9.2	18.3	+10.3	89.0	94.5	23.2	−5.5
variable tracking	62.0	82.2	14.2	24.4	−20.2	78.2	88.8	26.2	−10.6
common words	0.7	0.6	0.9	0.8	+0.1	3.8	26.6	2.5	−22.8
frequent words	76.0	74.7	77.7	73.7	+1.3	59.7	79.7	44.7	−20.0
question answering 1	41.0	60.0	20.0	22.0	−19.0	37.0	39.0	19.0	−2.0
question answering 2	38.0	37.0	31.0	31.0	+1.0	35.0	40.0	32.0	−5.0
aggregate (13 tasks)	**70.80**	69.86	14.77	21.91	+0.94	**67.80**	75.52	23.76	−7.72

5.4 Throughput and memory

Table 20 sets the design against stock llama.cpp on one H200. Below the trained range the design costs under ten percent of prefill and of steady decode, from the copy that streams rolled-off keys and values to the host and from the window-only prefill above 24.7K tokens; the target for that regime is parity, and it is not met. At 128K the design is faster than the dense model it runs on, nearly twice the prefill and about two thirds more steady decode, because every query attends over about 25K cells instead of 128K. Re-encodes at events are rare below 128K and a few per 128 decoded tokens there. The GPU cache is bounded by construction at the window, the far window and the pinned prefix, about 25K cells per layer at any length; the host store grows with the document, at 131 KB per token.

Table 20: Throughput against stock llama.cpp on one H200: prefill of the whole context and steady decode after it, tokens per second, with the design's ratio to stock, and the memory the design holds. Stock llama.cpp does not fit a 1M cache on one H200.

model	context	stock prefill / decode	design prefill	ratio	design decode	ratio	re-encodes / 128
Mistral-7B	8K	11,017 / 234.1	10,324	0.94	228	0.97	0
Mistral-7B	32K	8,850 / 180.2	8,084	0.91	184	1.02	0
Mistral-7B	128K	4,819 / 91.1	8,920	1.85	150	1.65	4
Llama-3.1-8B	8K	11,003 / 226.8	10,319	0.94	214	0.95	0
Llama-3.1-8B	32K	8,835 / 175.4	8,048	0.91	161	0.92	1
Llama-3.1-8B	128K	4,833 / 89.4	8,938	1.85	150	1.68	3
Mistral-7B	1M	does not fit	8,222		149		1

memory: the GPU cache is 24,705–25,728 cells per layer at every length (window, far window and pinned prefix); the host store is 131 KB per token at f16, 128.5 GiB of keys and values at 1M, and the process peaks at 132.6 GiB on the 1M needle runs

5.5 One million tokens

At one million tokens, 32 times Mistral-7B's trained range and eight times Llama-3.1-8B's, the design reads every single needle and all but one three-key discrimination, fetching a few thousand tokens per request. The engine prefills the million tokens at 8,222 tokens per second and decodes at 149, with a re-encode every hundred-odd generated tokens, and the host store holds the million tokens' keys and values in about 130 GiB. The sink must be kept exact through the moves of a million-token prefill (Section 4.2).

6 Matching on descriptions

Section 2.6 showed that attention cannot find anything but a word in the distant past of the attention window. A description does match, however, in the near window, and the word that match produces can reach far back into the history. We test this by asking the model to name the described thing before answering: the name it produces is a word in the near window, and that word is what reaches far back. Under dense attention both models produce the name in about three quarters of the cases, and the description read rises from about one in seven to about seven in ten (Table 21).

Under the design the same question takes four steps, because the sentence that holds the answer is not in the cache when the question is asked. The model generates the name. The name is a rare word, so the word index fetches the sentences it occurs in from the host store into the far window, the fetched sentence with the code among them. At that point the sentence is in the cache but unread: the request and the name were encoded before the sentence arrived, and Section 2.4 showed that under competition a far unit is read only if it was present when the question's handle word was encoded. So the engine rewinds the cache to the start of the request and rolls forward again over the request and the tokens generated so far, with the fetched sentences present. That pass is exact, byte-identical in its logits to a fresh prefill, and the code the model then produces is read from the fetched sentence. The harness shows the steps apart. With the far window holding only fetched units, the fetch without the roll-forward leaves Mistral-7B at almost nothing, since the sentence arrives among three same-format decoys; the roll-forward reads nearly two thirds; and with the fetch disabled both models read nothing, because the name is produced and nothing acts on it. Retrieval that fires on what the model generates is the active-retrieval idea [82], and naming before retrieving is the generate-then-retrieve idea [55]; what is specific here is that the generated word fetches into the model's own key-value cache, by the same index and the same slots as the request, and that the request is re-encoded over what arrives.

On the engine, Llama-3.1-8B at 128K reads about four fifths of what dense attention reads, and the verbatim handle nearly as often as dense attention. Mistral-7B at 32K reads well below dense attention and below the harness, and the gap to the harness is the far window. The engine's far window is always full (Section 3.2); at 32K its back-fill of the most recent far tokens covers about two thirds of the far region, so a same-format decoy is resident from the hand-off in nearly every item, and when the fetched sentence arrives and the request is re-encoded, the read goes to a resident decoy in most such rows, where the harness, whose far window held only fetched units, read the fetched sentence in nearly all of them. Denying the far window the same-format decoys recovers more than half of the gap, with the decoy reads gone and the remaining misses reading some other resident token; turning the re-encode off changes little, so the re-encode neither causes nor cures the drop. The cost is the far window's volume (Section 3.2) paid on the one read that competes on format rather than on words; measuring this match with a far window that holds only fetched units remains future work, and the Mistral-7B engine number is a property of the fill rather

than of the model or the store. On Llama-3.1-8B at 128K the back-fill covers a small part of the far region, few decoys are resident, and nearly every re-encoded row reads.

Native thinking does not replace the naming step. DeepSeek-R1-Distill-Llama-8B [40], asked the same questions under the design with its own reasoning trace, rarely names the target and never reads the code, and its verbatim read falls by a quarter once the trace is on; under dense attention the same model names the target about as often as the base models do and reads the code less often. The effective procedure is the short one: name, then answer. An extension of this procedure fetches the name's unit and glosses it into the request before encoding, which in the harness reads about as well as naming does on both models, with and without a worked example, well above its wrong-name controls. We leave this for future work.

Table 21: Matching on descriptions. Description-matched questions with three same-format decoys per document, $n = 81$, exact seven-digit match. "Harness" is the PyTorch reference, whose far window holds only fetched units; "engine" is the llama.cpp implementation, whose far window is always full. "Name first" asks the model to name the described thing before giving the code; "re-encode" is the roll-forward over the request and the generated tail after the fetch.

arm	Mistral-7B 32K	Llama-3.1-8B 128K
naming, under dense attention		
the description asked directly (harness)	9/81	13/81
name first, then the code (harness)	56/81	61/81
name first (engine)	59/81	64/81
the thing named at all (harness)	58/81	64/81
given a correct name		59/59
the design, name first		
events off (harness)	0/81	0/81
event fetch, no re-encode (harness)	2/81	44/81, 44 of the 51 units fetched
event fetch and re-encode (harness; the far window holds fetched units only)	51/81, past a bar of 41	not run
engine, the far window always full	36/81	50/81–51/81
same, same-format decoys denied from the far window (oracle)	45/81	
same, re-encode off	35/81	
the verbatim handle asked (engine; design / dense)	70/81 / 78/81	73/81 / 79/81
the thing named at all (engine; design / dense)	58/81 / 58/81	53/81 / 63/81
why the engine's Mistral-7B number is the fill		
a same-format decoy resident from the hand-off	73 of 81 items	few: the back-fill covers 16K of 123K far tokens
rows where the event fetched the unit and the request was re-encoded	the fetch loses to a resident decoy in 17 of 20 (harness: the fetched unit read in 50 of 51)	42 of 43 read
a native reasoning trace: DeepSeek-R1-Distill-Llama-8B, under the design / under dense attention		
names the target	13/81 / 66/81	
reads the code	0/81 / 43/81	
the verbatim read without / with the trace, under the design	80/81 / 60/81	
follow-up: the name's unit glossed into the request before encoding (harness; without / with a worked example)		
with the correct name	60/81 / 58/81	59/81 / 54/81
with a wrong name (control)	29/81 / 2/81	48/81 / 1/81

7 Limitations

The first hop limits two-hop questions. They reach parity with dense attention on Llama-3.1-8B at 128K and stay well below it on Mistral-7B at 32K (Table 18). The limit is the first hop, and in the engine it is the near window rather than the far window: at 32K the bridge word is produced about as often with the needle resident as by the window-only floor, and far more often under dense attention, so the near window alone loses most of the hop and the far window a little; at 128K the far window's share grows. On Llama-3.1-8B the answer is right far more often than the bridge word is produced, so the first-hop reading bounds Mistral-7B's result rather than Llama-3.1-8B's. Table 22 sorts the misses with the far seed off by cause: on Mistral-7B most are units resident without the bridge word being produced, and on Llama-3.1-8B most were handles outside the seed, a class the far seed removed. A pointer-aware read, or a second re-encode after the first hop's fetch, is a candidate remedy. It also suggests that the design is best applied to models trained to a longer range; a 32K-trained attention window can be stretched only so far.

Table 22: Two-hop misses by cause with the far seed off, $n = 100$ per row. The far seed removes the handle-outside-the-seed class on Llama-3.1-8B.

model	context	misses	what the misses were
Mistral-7B	32K	38	6 units not admitted; 23 resident without the bridge word; 9 resident with the word produced and misread
Mistral-7B	128K	45	11 not admitted; 25 resident without the bridge word; 4 wrong after a re-encode; 5 misread
Llama-3.1-8B	32K	26	14 handles outside the 32-word seed; 12 no bridge word
Llama-3.1-8B	128K	16	9 handles outside the seed; 7 no bridge word

The always-full far window costs reads. The far window back-fills to 16,384 slots with the most recent far tokens, and on reads that compete on format rather than on words that content costs: the description read through naming on Mistral-7B loses nearly a third of its reads against a far window of fetched units alone (Table 21), and two of the three multi-key RULER tasks on Mistral-7B at 32K lose about twenty points to dense attention (Table 19). Measuring the description match with a far window that holds only fetched units remains future work (Section 6).

The RULER results are mixed. The design's aggregate is a point above dense attention on Llama-3.1-8B at 128K and eight points below it on Mistral-7B at 32K (Table 19). The usual bar of 50 on every multi-key task is missed by three points on the third multi-key task; the tasks that chain through content, variable tracking and the first question-answering task, fall below dense attention on Llama-3.1-8B; the volume tasks, common and frequent words, fall below it on Mistral-7B, where the window-unit flag recovers part of the loss (Table 14). The matched floor, a 24,576-token window with no retrieval, scores far below the design on Llama-3.1-8B at 128K, so the aggregate comes from the far window rather than from the budget. The remaining controls the RULER bar asks for, a random store and the published training-free systems InfLLM [194] and ReAttention [114], were not run against the same items; and a needle that shares no word with its question, the NoLiMa setting [134], is outside what a lexical index can fetch without the naming step of Section 6.

Throughput below the trained range misses its target of parity with the stock engine. Prefill and steady decode each cost under ten percent there (Table 20), from the copy that streams rolled-off keys and values to the host and from the window-only prefill above 24.7K tokens.

The host store implemented in llama.cpp is unbounded and uncompressed, and holds about 130 GiB at one million tokens on Mistral-7B. This is a practical limit on serving an unbounded context. The compression of Section 3.2 cuts the store four and a half times, and beyond that the remedy

is a further tier: spill the store to disk and page units in as the index names them, which the word index already makes possible, since a unit is fetched by its words rather than scanned.

The scope is two 7–8B models of one architecture family, one item seed per row, and a 70B we measured only under full attention in the harness. Every model is rotary and dense, and the design refuses multi-head latent attention and linear-bias models by construction. Single-seed differences are read as parity, never as gains. The residency rule is measured on short answers; tokens that regain attention late in a long reasoning trace [216] are outside the measured regime, which is why the re-encode carries a per-turn budget. The census of Section 2.1 is over 2,048-token windows, and its sink share and far-content class depend on that length. The order-at-write half of the plane account rests on the writer-reader weights and a collapsed-position order question rather than on a write-time perturbation.

8 Discussion

We built the unbounded context for the straightforward use of a transformer model, one long document and a question at its end, but the concepts presented here extend to other uses. In this section we consider a few of the more fruitful directions.

The first direction is retrieval into the attention rather than into the prompt. Retrieval-augmented generation puts retrieved text into the prompt [77, 102, 156, 163]: an external index chooses passages, the passages are prepended as tokens, the model encodes them again at every request, and it reads them as text. The design here augments the attention instead. A span is pre-encoded through the model once and its keys and values are stored; the word index records its rare words; and when a request names one of them the span enters the far window as keys and values at a slot, where the model reads it as it would have read it in place. The measurement that licenses the first step is in Section 2.5: a forward pass over a unit's own tokens with no preceding context reads every needle from 32K to one million tokens, at a bulk cost of a few hundredths of a nat that a few hundred tokens of its own context removes (Table 10). So the context of the model need not be the conversation. A library, a code base, or a user's past sessions can be encoded once, offline, and become far context under the same index and the same far window, and the retrieved material then costs no prompt tokens, no re-encoding of the passages, and no place at the front of the prompt where order effects apply. The serving systems that precompute chunk caches for retrieval [2, 23, 62, 84, 103, 120, 126, 202, 205, 208] already build that machinery, most of them for prompt-scale contexts and with a fine-tune or a recompute halo that restores attention across chunk boundaries, and knowledge bases have been written into attention as key and value vectors through trained adapters [184] or a model trained for it [201]. What this paper adds to that program is where and when a pre-encoded span reads on a stock model: at any slot inside the trained range, degrading beyond about twice it; as the model's own keys and values rather than a synthesized stand-in; and resident when the question's handle word is encoded. Whether a span encoded without its document reads as well under competition as one captured from the window remains to be measured.

The second is that the retriever and the reader are decoupled. The word index is the retriever here, and it is a weak one on purpose: whole rare words, exact match, scored by surprisal. It is enough for every planted row, because a planted item has a handle, and the rows it loses on RULER are the two kinds a lexical index cannot serve: a question that shares no rare word with its answer, and many spans that share the question's words. NoLiMa is the benchmark form of the first [134], needles with no literal overlap with the question, and the description match of Section 6 is our answer to it: the model names the thing, and the name is the key. A better scorer is the answer to

the second, and the far window is the place to measure one, since any ranking that produces units can enter it by the same placement: a dense retriever's top units, a hybrid of words and vectors, or the model's own keys as the key-similarity family uses them [27, 109, 194]. The attention performs the final read in every case, so a far window holding only fetched units isolates the reader from the retriever and is the natural next measurement.

The third is that timing is a rule for every retrieval system. The residency finding says more than fetch first. The binding forms at the handle word's own query, and a unit that arrives later is unread until the question is encoded over it. For a prompt-side pipeline that is the rule that retrieved text precedes the question, which practice already follows. For a system that retrieves while it generates [6, 82, 170] it says that the retrieved material must be encoded together with the question, which the prompt route pays for with the whole prompt and the attention route pays for with one exact re-encode, a re-run of the request and the generated tail over what arrived, a few tens of milliseconds here (Table 14). An agent that calls a tool mid-generation and reads the result faces the same rule, and the event mechanism is one way to meet it inside the cache. The cost side has a lesson as well: the gate on the generated word's surprisal cut the re-encodes to a few per hundred generated tokens without losing a read, so the model's own uncertainty about a word is an adequate trigger, as the active-retrieval line found on the prompt side.

The fourth is a memory that outlives the session. The host store is position-free and exact, so nothing in it depends on the session that wrote it, and resuming a session from its store is a test the implementation has not run. A store that persists is a memory of everything the model has read, with the index as its catalog, and the question it raises is the question that usage-based eviction cannot answer: what to forget. On a document the answer was nothing, because no far unit is used before its question. On a lifetime store the criterion is open; age, source and index demand are the candidates, and the measurement to make is the same three-key discrimination under each. The costs are concrete. At 131 KB per token a terabyte of host memory holds about eight million tokens exact; at the 29 KB of the measured 28-plane four-bit rung it holds about 35 million; the gather moves a few thousand tokens per request at one million, and the GPU working set is fixed at the near window, the far window and the pins. The memory wall moves from the device to the interconnect, and the traffic across it is a few thousand cells per request, small against what the link carries. Agent memories kept as keys and values rather than as text [146, 149] are the same idea seen from the application side.

The fifth is the line between what the stock model does and what training adds. Every read here is lexical, a description does not cross the window, and a second hop is work done while generating. Those are properties of the models as trained, and the retrofit exposes them rather than changing them. Models trained to longer contexts [57, 198] buy a longer window; what they buy for a far read is unmeasured here, and the census suggests the answer is the same handful of reader heads at a longer reach. The more interesting question is the reverse one: a model fine-tuned with a far window present, so that it learns to read fetched units by what they mean rather than by the words they share with the question, would move the description read into the retrieval tier. The trained-memory line [135, 137, 192] has built such readers from scratch; whether one can be grafted onto a stock model with a few steps of training, while the window and the sink stay as they are, is a question the engine makes testable. More generally, Section 2 presents a functional view of what attention does. A model and a training regime aimed at those functions, rather than left to discover them from a generic initialization, could serve them better; that is a direction for training rather than for serving.

The last direction is serving less of the cache. Half of the attention mass is routing within 64 tokens, a few mid-stack heads do the far reading, and most of the rest is parked on the sink. The design ignores that: one store, one index, one far window, read by every layer alike. A design that

kept the far window only for the layers that read it, or scheduled the gather by which heads will fire, would cut the GPU cache below its present bound, and the head-level cache designs [53, 176, 196] show the magnitude of the available saving. The selection itself stays lexical in such a design, since a content match does not make a far unit readable (Section 2.6); the saving comes from serving the reads where they occur.

9 Related work

The design sits at the meeting point of several literatures: streaming attention and its sinks, caches that store keys without a position, retrieval over the cache, eviction and salience rules, trained memories, the reuse of precomputed caches for retrieval, the timing of a read, the structure of rotary attention, the heads that copy, and retrieval that fires while a model generates. Each block below names the work our design stands on and states what we add to it.

Streaming windows and sinks. StreamingLLM keeps a near window and the first tokens as attention sinks and shows that removing the sinks is catastrophic [195]; LM-Infinite reaches the same window-plus-anchor shape through a Λ-shaped mask and a distance ceiling [65]. The sinks are the massive activations of Sun et al. [173]; Gu et al. [64] trace when they emerge in training, Barbero et al. [11] read first-token attention as a control on over-mixing, and KVSink preserves them under quantization [171]. SepLLM keeps separators as segment summaries [25], and Star Attention prefixes an anchor block to every block so that each has a sink to stand on [1]. What is specific here is where the sink must sit: at a slot inside the trained range, since at its true distance it costs +0.58 and +1.15 nats by one million tokens, and the drift of a sink key moved in place about 8,100 times in f16, which the engine fixes by re-placing the pinned prefix from the host. The census adds the sink's share of the mass per layer and its reading as the softmax floor rather than a read.

Position-free storage and re-placement. Storing keys without a position and rotating them at a new one is the primitive of CachedAttention [54], EPIC [75], MiniPIC [143], LazyAttention [193] and Kamera [125], and CacheFocus re-positions cached document chunks into the trained length for retrieval [100]. ReAttention selects far tokens without position and rotates the selected ones into a finite scope, training-free, on these same models [114]; Extended Mind Transformers retrieve the model's own cached keys and values by similarity and find that no position at all serves them best under rotary embeddings [93]; Parallel Context Windows reuse one range of positions for several windows [157]. SelKV reports that re-embedding kept keys at contiguous positions can hurt a long-context read [18], which our tolerance measurement bounds. The length-extension line moves the positions instead of the keys: interpolation [26], per-band scaling [44, 150, 161], grouped far positions [85], chunked remapping [4], a shift over the under-trained tail [5] and a clip of the slow components [104]; the trained extensions of Gao et al. [57] and Xu et al. [198] buy the range outright. The measurement added here is the tolerance of the far match: any slot inside the trained range, a slow degradation to about twice it, failure at distance zero and at the sink, and a GPU cache bounded by a word index rather than by the document.

Retrieval over the cache and sparse attention. The training-free systems closest in shape keep a sink, a window and far units in host memory and fetch units by a similarity between the query and a representative of the unit: InfLLM [194], LongHeads [121], RetrievalAttention and

RetroInfer [27, 109], MagicPIG [28], PQCache [215], ShadowKV [172], ClusterKV [110], TokenSelect [190], InfiniGen [101], HeadInfer [122] and InfiniteHiP, which reaches three million tokens on one GPU [98]. Within the trained length, Quest [177], MInference [80], SparQ [158], Loki [165], HashAttention [41], Squeezed Attention [71], TidalDecode [203], DuoAttention [196] and RazorAttention [176] select blocks or heads by key statistics at run time; Multipole Attention and CEDAR score centroids or chunk summaries inside one softmax [72, 105], and vAttention bounds the error of such a selection [42]. EM-LLM segments the context into events by surprise and retrieves them by similarity and contiguity [52], and S^3-Attention keeps a CPU inverted index of learned sparse features and fuses BM25 matching for rare entities [127], the nearest lexical neighbor of the index here. Native Sparse Attention, MoBA and the sparse attention of DeepSeek-V3.2 train the selection in [39, 119, 213]. The retrofit here selects by words rather than by keys, needs nothing trained, and places whole sentence units into a far window whose tolerance is measured.

Eviction, usage and salience. Usage-scored stores run from the least-recently-used memory of Santoro et al. [160] to heavy-hitter eviction [222], the persistence-of-importance hypothesis of Scissorhands [117], the observation window of SnapKV [106], the layer budgets of PyramidKV [22], TOVA [144], cascading caches [189] and forgetting factors [86]; learned retention gates follow [19, 20, 204], and Steele [168] and Kumar and Mishra [94] measure where learned importance scoring stops working. Attendre keeps evicted queries so that a key can still be attended by a query that arrives later [206], the nearest precedent for the timing problem of Section 2.4. Lexical and salience rules decide what to retain in Transactional Attention [12], on these same models, in SirLLM [209], SALT [129] and Pulsar Attention [167], TriRoute learns the routing [9], and BudgetMem filters chunks by lexical statistics on the prompt side [3]. The admission rule here is a member of that family, and what we add to it is the demand bound at one million tokens and the finding that usage-at-exit eviction of a host store fails on a long document, because no far unit is used before its question.

Trained memories and compressed contexts. Memorizing Transformers [192], Landmark Attention [135], Unlimiformer [15], the Focused Transformer [180], LongMem [183], MemLong [113], Infini-attention [137], Transformer-XL [36] and the kNN language model [90] train for a memory; a reproduction of Infini-attention reports that its compressed memory struggles to retrieve from earlier segments [138]. Compression of the context into trained tokens or states [30, 58, 136, 155, 217] and latent memory pools [188] trade the exact history for a smaller one, and RetroLM retrieves cache pages with a trained page retriever [124]. The retrofit here trains nothing and keeps the far history exact; its measurement that a synthesized key or value reads nothing while a self-consistent forward over the unit's own text reads 14/14 at one million tokens is the reason the store holds the model's own cells.

Reusing precomputed caches for retrieval. Prompt Cache reuses the keys and values of prompt modules at schema-assigned positions [62], and the retrieval-augmented serving line precomputes a cache per chunk and concatenates the chunks a request retrieves: TurboRAG [120], Block-Attention [126] and KVLink [202] fine-tune the model to the independently encoded chunks, APE aligns them without training through a shared prefix, a scale and a temperature [205], and CacheBlend [208], Cache-Craft [2], EPIC [75], Star Attention [1] and Mao et al. [130] recompute a small set of tokens so that reused chunks read correctly, while Tachibana et al. [175] combine a concatenation-aware fine-tune with recomputation and report the one RULER number in that line past 100K tokens. Cache-augmented generation preloads a whole knowledge base as a cache inside

the window [23], REFRAG feeds compressed chunk embeddings to the decoder and expands the ones a policy selects [108], Li [103] precompile chunks and re-rotate their keys before splicing them in, and agent memories are kept as caches rather than as text [149]. Serving systems supply the tiers such caches live in [56, 115, 116, 154, 223], and KVShareArena benchmarks the reuse [164]. What is added here is the unit-level result at range on a stock model: a span rebuilt from its own tokens with no preceding context reads 14/14 at one million tokens where a higher-fidelity synthesized key reads nothing, so a self-consistent forward pass outperforms a better map, and the slot it is placed at may be anywhere inside the trained range.

Residency and query visibility. Lost in the Middle moves the question to show that content encoded with the question visible is read better [112]; Luo et al. [123] show that compression rankings invert with the scorer's query visibility, WnW that prefill and decode rankings overlap weakly [210], and KVzip, Lookahead Q-Cache, DapQ and Expected Attention estimate the queries a cache will face before they arrive [43, 92, 178, 187]. LazyEviction shows tokens regaining attention late in long reasoning traces [216], Retrospective Sparse Attention revises earlier outputs when later keys arrive [32], and Okamoto and Sarti [141] find a signal encoded early and acted on late. The residency measurements here hold content, order and attention mass fixed and vary only the moment a far unit becomes visible, which locates the binding at the handle word's own query under competition and motivates the exact re-encode at an event, with a per-turn budget for the regime LazyEviction describes.

Rotary planes and key compression. The semantic and positional roles of rotary frequencies [10, 69, 150, 169] are the ground of this paper's plane account; Chiang and Yogatama [31] find that long-distance retrieval rides on the lowest-frequency dimensions, Oka et al. [140] locate a high-norm band of frequencies whose place depends on the base, Men et al. [133] bound the context length by the base, and Du et al. [45] prove what rotary attention can and cannot distinguish at long range. The analytic per-wavelength window of ATFlash [67] is the model-card plane cut used here; the frequency-selecting compressions EliteKV, RAP, FASA and KV-Latent [162, 186, 197, 225] and the key-channel pruning of ThinK, LeanK, Prism and Thin Keys, Full Values [185, 200, 207, 221] cut the same dimensions by a learned or measured score; KIVI, KVQuant, NSNQuant and TriAttention quantize or compress the cache [70, 118, 131, 166]. The plane results here are the measured cliff on multi-candidate reads between 22 and 15 planes, the fact that loss cannot see it, and the sink's destruction when only fast planes are kept; the design refuses multi-head latent attention [38] and linear-bias schemes [151], which have no per-plane rotation to place.

Heads, induction and copying. Head-role studies [34, 181, 224] and induction heads [16, 46, 142] are the classic accounts; PAPA asks how much attention is needed at all [66], and Chang and Bergen [24] map the bigram subnetwork that next-token prediction rests on. Retrieval heads [13, 53, 97, 191, 218], binding identifiers [48], information flow routes [49] and factual recall [60] locate the copy, Future Lens reads its carry [147], copy suppression is its counterweight [132], and Hayes et al. [68] and Huang et al. [76] synthesize a program per head or per model. The dual-route model separates a verbatim from a semantic induction route by ablation [51]; D'Angelo et al. [37] show on synthetic data that induction weights a context match exponentially by its overlap, and the generalized induction head of Kim et al. [91] adds fuzzy matching to the count. The aligned-match result of Section 2.1 is the deployment-scale form of that finding, on a 7B model reading prose. The census here is a pooled mass accounting over every head with the sinks reported separately; its text-only chooser is of the suffix kind that infini-gram scales to a trillion tokens [111]; and it

stays behavioral: no head is ablated and no head is replaced.

Values and words. That a multi-token word is assembled early in the residual stream is the finding of Feucht et al. [50], Kaplan et al. [89], Kamoda et al. [87] and Busigin and Pinter [21], and the staged picture of inference of Lad et al. [96]; the lenses that read the residual [14, 61] and the additive account of recall [33] are the instruments. On the cache side, KV Prediction approximates keys and values for a faster first token [73] and AsymKV keeps the values lossless because they tolerate less [35]. What we add is the serving consequence: a synthesized value destroys the read while a synthesized key only misaddresses it, and a word's assembled vector is order-coded and readable through the embedding table where the logit lens is not.

Retrieval during generation. FLARE retrieves when a generated span is uncertain [82], DRAGIN and SeaKR read the model's own information need [170, 211], Self-RAG trains reflection tokens that decide to retrieve [6], Adaptive-RAG routes by question complexity [78], and Zhang and Pei [219] prefetch what a generation will ask for. IRCoT interleaves retrieval with reasoning steps [179], Self-Ask decomposes the question [152], and HyDE, Query2doc and query rewriting generate before they retrieve [55, 128, 182]; MemoRAG drafts clues from a global memory [153], and MemGPT and ReadAgent page text in and out of the context [99, 146]. Retrieval augmentation itself [17, 102, 156, 163] and its comparison with long-context models [83, 107] frame the question Section 8 takes up. The events and the description match are that idea inside the model's own cache: a generated rare word fetches its unit by the same index and into the same slots as the request, and the re-encode is exact.

Evaluation. The needle protocol is Kamradt's [88]; RULER supplies the generator, the tasks and the metric [74]; LongBench, ∞Bench, HELMET, LV-Eval and BABILong extend the evaluation to realistic tasks and to millions of tokens [7, 8, 95, 212, 214, 220], Needle Threading follows chains through near-million-token haystacks [159], and NoLiMa removes the literal match between question and needle [134], the test a lexical index must eventually face. Long-range loss is known to be a poor guide to retrieval: Sun et al. [174] found long-range models barely use far context except for copy-like tokens, Fang et al. [47] show perplexity missing long-context ability, and retrieval helps the rare tokens that bulk loss barely weighs [139], which is why every store change here is validated on three-key discrimination rather than on loss. Prompt compression [29, 81, 148, 199] is the prompt-side alternative to keeping the far text exact.

10 Conclusion

This paper began by examining the geometry and function of attention in a stock rotary model. A far key is addressed through its slowest rotary planes, which is why it can sit at any position inside the trained range and still be found. The order of the tokens inside a span is fixed when their keys are written, so a span can be moved after the fact without losing it. A far read is a copy: a few heads in the middle of the stack find an earlier occurrence of the current word and copy what followed it, which is why a far span is reached by its words rather than by its meaning. When several spans compete, the one that is read is the one present at the moment the question's handle word is encoded, so a fetched span must arrive before the question and stay until the answer is produced. The sink and the anchors hold the softmax floor only while they sit inside the trained range.

A stock decoder with those facts applied reads its whole history through a bounded GPU cache, on the checkpoint and the runtime already in use, at one million tokens on one device, with every setting but one read off the model card. The far window that makes needle reads perfect is always full, and its back-fill is what the one read that competes on format rather than on words pays for; a far window of fetched units alone remains to be measured. The first hop of a two-hop question on a 7B model is lost by the near window before any retrieval begins, which points at a second re-encode after the first hop's fetch, or at a read that follows a pointer. A description is no address until the model has turned it into a word, and the model can do that in its own output: matching on descriptions through the model's own naming is the start of a retrieval loop in which the model writes its own queries. Beyond the host horizon a unit's keys can be rebuilt from its text, so the history need not even be stored as keys, and anything the model has once encoded can enter its attention the same way, which is the retrieval Section 8 describes. The deepest open question is the one the census raised: half of attention is routing, a few heads do the reading, and the rest is mass parked on anchors. A serving design that knows which heads read, and what they read by, could reduce the GPU cache further.

References

[1] Shantanu Acharya, Fei Jia, and Boris Ginsburg. Star attention: Efficient LLM inference over long sequences. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2411.17116.

[2] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*, 2025. arXiv:2502.15734.

[3] Chandra Vamsi Krishna Alla, Harish Naidu Gaddam, Manohar Kommi, and Sheikh Nazib Ahmed. BudgetMem: Training-free selective memory for cost-efficient long-context processing in language models. *arXiv preprint arXiv:2511.04919*, 2025.

[4] Chenxin An, Fei Huang, Jun Zhang, Shansan Gong, Xipeng Qiu, Chang Zhou, and Lingpeng Kong. Training-free long-context scaling of large language models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.17463.

[5] Chenxin An, Jun Zhang, Ming Zhong, Lei Li, Shansan Gong, Yao Luo, Jingjing Xu, and Lingpeng Kong. Why does the effective context length of LLMs fall short? In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.18745.

[6] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[7] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2308.14508.

[8] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2412.15204.

[9] Andrii Balashov and Olena Ponomarova. TriRoute: Unified learned routing for joint adaptive attention, experts, and KV-cache allocation. *arXiv preprint arXiv:2607.06601*, 2026.

[10] Federico Barbero, Alex Vitvitskyi, Christos Perivolaropoulos, Razvan Pascanu, and Petar Veličković. Round and round we go! what makes rotary positional encodings useful? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2410.06205.

[11] Federico Barbero, Álvaro Arroyo, Xiangming Gu, Christos Perivolaropoulos, Michael Bronstein, Petar Veličković, and Razvan Pascanu. Why do LLMs attend to the first token? *arXiv preprint arXiv:2504.02732*, 2025.

[12] Abhinaba Basu. Transactional attention: Semantic sponsorship for KV-cache retention. *arXiv preprint arXiv:2604.11288*, 2026.

[13] Cengizhan Bayram. Does RoPE prevent or degrade retrieval heads? a mechanistic analysis across model families. *arXiv preprint arXiv:2606.21249*, 2026.

[14] Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

[15] Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. Unlimiformer: Long-range transformers with unlimited length input. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.01625.

[16] Alberto Bietti, Vivien Cabannes, Diane Bouchacourt, Herve Jegou, and Leon Bottou. Birth of a transformer: A memory viewpoint. In *Advances in NeurIPS*, 2023. arXiv:2306.00802.

[17] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2112.04426.

[18] Soumia Bouyahiaoui, Manel Kara laouar, Aicha Boutorh, and Mohamed Hadj Ameur. SelKV: Selective KV cache merging with per-token merge-or-drop and attention compensation. *arXiv preprint arXiv:2607.16213*, 2026.

[19] Ngoc Bui, Shubham Sharma, Simran Lamba, Saumitra Mishra, and Rex Ying. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. *arXiv preprint arXiv:2512.03324*, 2025.

[20] Ngoc Bui, Hieu Trung Nguyen, Arman Cohan, and Rex Ying. Make each token count: Towards improving long-context performance with KV cache eviction. *arXiv preprint arXiv:2605.09649*, 2026.

[21] Benzi Busigin and Yuval Pinter. Inside the LLM word factory. In *Findings of EMNLP*, 2026. arXiv:2606.08562.

[22] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

[23] Brian J Chan, Chao-Ting Chen, Jui-Hung Cheng, and Hen-Hsen Huang. Don't do RAG: When cache-augmented generation is all you need for knowledge tasks. In *Proceedings of the ACM Web Conference (WWW) (Companion, short paper)*, 2025. arXiv:2412.15605.

[24] Tyler A. Chang and Benjamin K. Bergen. Bigram subnetworks: Mapping to next tokens in transformer language models. In *Advances in NeurIPS*, 2025. arXiv:2504.15471.

[25] Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. SepLLM: Accelerate large language models by compressing one segment into one separator. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2412.12094.

[26] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

[27] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jing Liu, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Cheng Li, Yuqing Yang, Fan Yang, and Mao Yang. RetroInfer: A vector storage engine for scalable long-context LLM inference. *Proceedings of the VLDB Endowment*, 19(5):1016–1031, 2026. arXiv:2505.02922.

[28] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. MagicPIG: LSH sampling for efficient LLM generation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.16179.

[29] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.13792.

[30] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.14788.

[31] Ting-Rui Chiang and Dani Yogatama. The rotary position embedding may cause dimension inefficiency in attention heads for long-distance retrieval. *arXiv preprint arXiv:2502.11276*, 2025.

[32] Seonghwan Choi, Beomseok Kang, Dongwon Jo, and Jae-Joon Kim. Retrospective sparse attention for efficient long-context generation. *arXiv preprint arXiv:2508.09001*, 2025.

[33] Bilal Chughtai, Alan Cooney, and Neel Nanda. Summing up the facts: Additive mechanisms behind factual recall in LLMs. *arXiv preprint arXiv:2402.07321*, 2024.

[34] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP*, 2019. arXiv:1906.04341.

[35] Wanyun Cui and Mingwei Xu. Homogeneous keys, heterogeneous values: Exploiting local KV cache asymmetry for long-context LLMs. In *Advances in NeurIPS*, 2025. arXiv:2506.05410.

[36] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. arXiv:1901.02860.

[37] Francesco D'Angelo, Oguz Kaan Yuksel, Swathi Shree Narashiman, and Nicolas Flammarion. Induction heads interpolate n-grams. In *Proceedings of ICML*, 2026. arXiv:2607.02800.

[38] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[39] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[40] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[41] Aditya Desai, Shuo Yang, Alejandro Cuadron, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. HashAttention: Semantic sparsity for faster inference. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2412.14468.

[42] Aditya Desai, Kumar Krishna Agrawal, Shuo Yang, Alejandro Cuadron, Luis Gaspar Schroeder, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. vAttention: Verified sparse attention. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.05688.

[43] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025.

[44] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. LongRoPE: Extending LLM context window beyond 2 million tokens. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.13753.

[45] Yufeng Du, Phillip Harris, Minyang Tian, Eliu A Huerta, Srikanth Ronanki, Subendhu Rongali, Aram Galstyan, and Hao Peng. RoPE distinguishes neither positions nor tokens in long contexts, provably. *arXiv preprint arXiv:2605.15514*, 2026.

[46] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

[47] Lizhe Fang, Yifei Wang, Zhaoyang Liu, Chenheng Zhang, Stefanie Jegelka, Jinyang Gao, Bolin Ding, and Yisen Wang. What is wrong with perplexity for long-context language modeling? In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.23771.

[48] Jiahai Feng and Jacob Steinhardt. How do language models bind entities in context? *arXiv preprint arXiv:2310.17191*, 2024.

[49] Javier Ferrando and Elena Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.

[50] Sheridan Feucht, David Atkinson, Byron Wallace, and David Bau. Token erasure as a footprint of implicit vocabulary items in LLMs. *arXiv preprint arXiv:2406.20086*, 2024.

[51] Sheridan Feucht, Eric Todd, Byron C. Wallace, and David Bau. The dual-route model of induction. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2504.03022.

[52] Zafeirios Fountas, Martin A. Benfeghoul, Adnan Oomerjee, Fenia Christopoulou, Gerasimos Lampouras, Haitham Bou-Ammar, and Jun Wang. Human-inspired episodic memory for infinite context LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.09450.

[53] Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level KV cache compression method with integrated retrieval and reasoning. In *ICLR*, 2025. arXiv:2410.19258.

[54] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (USENIX ATC)*, 2024. arXiv:2403.19708.

[55] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2212.10496.

[56] Shiwei Gao, Youmin Chen, and Jiwu Shu. Fast state restoration in LLM serving with HCache. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2410.05004.

[57] Tianyu Gao, Alexander Wettig, Howard Yen, and Danqi Chen. How to train long-context language models (effectively). In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2410.02660.

[58] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.06945.

[59] Georgi Gerganov and llama.cpp contributors. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023. KV-cache position shifting via `llama_kv_cache_seq_add` and the `-context-shift` flag.

[60] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2304.14767.

[61] Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *Proceedings of ICML*, 2024. arXiv:2401.06102.

[62] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2311.04934.

[63] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[64] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When attention sink emerges in language models: An empirical view. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10781.

[65] Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. LM-Infinite: Zero-shot extreme length generalization for large language models. In *Proceedings of the Conference of the North American Chapter of the ACL (NAACL)*, 2024. arXiv:2308.16137.

[66] Michael Hassid, Hao Peng, Daniel Rotem, Jungo Kasai, Ivan Montero, Noah A. Smith, and Roy Schwartz. How much does attention actually attend? questioning the importance of attention in pretrained transformers. In *Findings of EMNLP*, 2022. arXiv:2211.03495.

[67] Shun-ichiro Hayashi, Daichi Mukunoki, Tetsuya Hoshino, and Takahiro Katagiri. ATFlash: Per-RoPE-wavelength attention windows for compute/memory-efficient LLM inference. *arXiv preprint arXiv:2608.02947*, 2026.

[68] Amiri Hayes, Belinda Z. Li, and Jacob Andreas. Explaining attention with program synthesis. *arXiv preprint arXiv:2606.19317*, 2026.

[69] Xiangyu Hong, Che Jiang, Biqing Qi, Fandong Meng, Mo Yu, Bowen Zhou, and Jie Zhou. On the token distance modeling ability of higher RoPE attention dimension. In *Findings of the Association for Computational Linguistics: EMNLP*, 2024. arXiv:2410.08703.

[70] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in NeurIPS*, 2024. arXiv:2401.18079.

[71] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Monishwaran Maheswaran, Sebastian Zhao, June Paik, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Squeezed attention: Accelerating long context length LLM inference. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2411.09688.

[72] Coleman Hooper, Sebastian Zhao, Luca Manolache, Sehoon Kim, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Multipole attention for efficient long context reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.13059.

[73] Maxwell Horton, Qingqing Cao, Chenfan Sun, Yanzi Jin, Sachin Mehta, Mohammad Rastegari, and Moin Nabi. KV prediction for improved time to first token. *arXiv preprint arXiv:2410.08391*, 2024.

[74] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024. arXiv:2404.06654.

[75] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332*, 2024. The algorithm is LegoLink, not AttnLink. From the v3 HTML: "LegoLink recomputes k (k<=32) initial tokens of each chunk (except the first chunk)", which cripples the spurious attention-sink at every chunk start.

[76] Xinting Huang, Aleksandra Bakalova, Satwik Bhattamishra, William Merrill, and Michael Hahn. Discovering interpretable algorithms by decompiling transformers to RASP. In *Proceedings of ICML*, 2026. arXiv:2602.08857.

[77] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 2023. arXiv:2208.03299.

[78] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of NAACL*, 2024. arXiv:2403.14403.

[79] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[80] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.02490.

[81] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2310.06839.

[82] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.06983.

[83] Ziyan Jiang, Xueguang Ma, and Wenhu Chen. LongRAG: Enhancing retrieval-augmented generation with long-context LLMs. *arXiv preprint arXiv:2406.15319*, 2024.

[84] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.

[85] Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. LLM maybe LongLM: Self-extend LLM context window without tuning. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.01325.

[86] Hyun-rae Jo and Dongkun Shin. A2SF: Accumulative attention scoring with forgetting factor for token pruning in transformer decoder. *arXiv preprint arXiv:2407.20485*, 2024.

[87] Go Kamoda, Benjamin Heinzerling, Tatsuro Inaba, Keito Kudo, Keisuke Sakaguchi, and Kentaro Inui. Weight-based analysis of detokenization in language models: Understanding the first stage of inference without inference. In *Findings of NAACL*, 2025. arXiv:2501.15754.

[88] Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[89] Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. From tokens to words: On the inner lexicon of LLMs. In *ICLR*, 2025. arXiv:2410.05864.

[90] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1911.00172.

[91] Eunji Kim, Sriya Mantena, Weiwei Yang, Chandan Singh, Sungroh Yoon, and Jianfeng Gao. Interpretable next-token prediction via the generalized induction head. In *Advances in NeurIPS*, 2025. arXiv:2411.00066.

[92] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in NeurIPS*, 2025. arXiv:2505.23416.

[93] Phoebe Klett and Thomas Ahle. Extended mind transformers. *arXiv preprint arXiv:2406.02332*, 2024.

[94] Pankaj Kumar and Subhankar Mishra. PAGE: Partition-aware gated KV-cache eviction. *arXiv preprint arXiv:2609.22157*, 2026.

[95] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *Advances in Neural Information Processing Systems (NeurIPS) (Datasets and Benchmarks)*, 2024. arXiv:2406.10149.

[96] Vedang Lad, Jin Hwa Lee, Wes Gurnee, and Max Tegmark. The remarkable robustness of LLMs: Stages of inference? *arXiv preprint arXiv:2406.19384*, 2024.

[97] Changhun Lee, Minsang Seok, Jun-gyu Jin, Younghyun Cho, and Eunhyeok Park. SEAL: Scaling to emphasize attention for long-context retrieval. In *Proceedings of ACL*, 2025. arXiv:2501.15225.

[98] Heejun Lee, Geon Park, Jaduk Suh, and Sung Ju Hwang. InfiniteHiP: Extending language model context up to 3 million tokens on a single GPU. *arXiv preprint arXiv:2502.08910*, 2025.

[99] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.09727.

[100] Kun-Hui Lee, Eunhwan Park, Donghoon Han, and Seung-Hoon Na. CacheFocus: Dynamic cache re-positioning for efficient retrieval-augmented generation. *arXiv preprint arXiv:2502.11101*, 2025.

[101] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. arXiv:2406.19707.

[102] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.

[103] Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026.

[104] Haoran Li, Sucheng Ren, Alan Yuille, and Feng Wang. CoPE: Clipped RoPE as a scalable free lunch for long context LLMs. *arXiv preprint arXiv:2602.05258*, 2026.

[105] Siyu Li, Dong Wang, Jie Zhou, Wei Li, Yang Xu, and Sijie Song. CEDAR: Error-bounded residual routing for efficient long-context attention. *arXiv preprint arXiv:2609.07237*, 2026.

[106] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[107] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. Retrieval augmented generation or long-context LLMs? a comprehensive study and hybrid approach. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP) (Industry)*, 2024. arXiv:2407.16833.

[108] Xiaoqiang Lin, Aritra Ghosh, Bryan Kian Hsiang Low, Anshumali Shrivastava, and Vijai Mohan. REFRAG: Rethinking RAG based decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2509.01092.

[109] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2409.10516.

[110] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.

[111] Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2401.17377.

[112] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024. arXiv:2307.03172.

[113] Weijie Liu, Zecheng Tang, Juntao Li, Kehai Chen, and Min Zhang. MemLong: Memory-augmented retrieval for long text modeling. *arXiv preprint arXiv:2408.16967*, 2024.

[114] Xiaoran Liu, Ruixiao Li, Qipeng Guo, Zhigeng Liu, Yuerong Song, Kai Lv, Hang Yan, Linlin Li, Qun Liu, and Xipeng Qiu. ReAttention: Training-free infinite context with finite attention scope. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.15176.

[115] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM Conference*, 2024. arXiv:2310.07240.

[116] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[117] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.17118.

[118] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[119] Enzhe Lu et al. MoBA: Mixture of block attention for long-context LLMs. *arXiv preprint arXiv:2502.13189*, 2025.

[120] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2410.07590.

[121] Yi Lu, Xin Zhou, Wei He, Jun Zhao, Tao Ji, Tao Gui, Qi Zhang, and Xuanjing Huang. LongHeads: Multi-head attention is secretly a long context processor. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. arXiv:2402.10685.

[122] Cheng Luo, Zefan Cai, Hanshi Sun, Jinqi Xiao, Bo Yuan, Wen Xiao, Junjie Hu, Jiawei Zhao, Beidi Chen, and Anima Anandkumar. HeadInfer: Memory-efficient LLM inference by head-wise offloading. *arXiv preprint arXiv:2502.12574*, 2025.

[123] Daming Luo, Christy Liang, and Junyu Xuan. How query visibility changes KV-cache compression rankings: A matched-budget audit. *arXiv preprint arXiv:2607.11942*, 2026.

[124] Kun Luo, Zheng Liu, Peitian Zhang, Hongjin Qian, Jun Zhao, and Kang Liu. Does RAG really perform bad for long-context processing? *arXiv preprint arXiv:2502.11444*, 2025.

[125] Bole Ma, Jan Eitzinger, Harald Koestler, and Gerhard Wellein. Kamera: Unified position-invariant multimodal KV cache for training-free reuse. *arXiv preprint arXiv:2606.23581*, 2026.

[126] Dongyang Ma, Yan Wang, and Lan Tian. Block-attention for efficient prefilling. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15355.

[127] Qingsen Ma, Dianyun Wang, Yaoye Wang, Lechen Ning, Sujie Zhu, Xiaohang Zhang, Jiaming Lyu, Linhao Ren, Zhenbo Xu, and Zhaofeng He. S^3-Attention: Attention-aligned endogenous retrieval for memory-bounded long-context inference. *arXiv preprint arXiv:2601.17702*, 2026.

[128] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models. In *Proceedings of EMNLP*, 2023. arXiv:2305.14283.

[129] Oteo Mamo, Hyunjin Yi, Joydhriti Choudhury, Shangqian Gao, and Weikuan Yu. SALT: Salience-aware lexical trie for long-context compression. *arXiv preprint arXiv:2607.17486*, 2026.

[130] Mingyang Mao, Wyatt Mackey, and Xiaomin Lin. Contiguity, not importance: Budgeted repair of stale KV caches after document edits. *arXiv preprint arXiv:2609.17983*, 2026. 16 Sep 2026. Frames in-place repair as budgeted recomputation and finds "a contiguous edit-local window recovers at least 0.94 of the post-edit answer margin", beating attention-based and structural selectors; 13-21x faster than full re-prefill. Closest published statement that the right recompute set is a contiguous span of neighbours, not a scattered important set.

[131] Weian Mao, Xi Lin, Wei Huang, Yuxin Xie, Tianfu Fu, Bohan Zhuang, Song Han, and Yukang Chen. TriAttention: Efficient long reasoning with trigonometric KV compression. *arXiv preprint arXiv:2604.04921*, 2026.

[132] Callum McDougall, Arthur Conmy, Cody Rushing, Thomas McGrath, and Neel Nanda. Copy suppression: Comprehensively understanding an attention head. *arXiv preprint arXiv:2310.04625*, 2023.

[133] Xin Men, Mingyu Xu, Bingning Wang, Qingyu Zhang, Hongyu Lin, Xianpei Han, and Weipeng Chen. Base of RoPE bounds context length. *arXiv preprint arXiv:2405.14591*, 2024.

[134] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-context evaluation beyond literal matching. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.05167.

[135] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16300.

[136] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.08467.

[137] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with Infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.

[138] Phuc Nguyen, Leandro von Werra, and Thomas Wolf. A failed experiment: Infini-Attention, and why we should keep trying? Hugging Face blog, https://huggingface.co/blog/infini-attention, 2024. Published 14 Aug 2024.

[139] Yuto Nishida, Makoto Morishita, Hiroyuki Deguchi, Hidetaka Kamigaito, and Taro Watanabe. Long-tail crisis in nearest neighbor language models. In *Findings of NAACL*, 2025. arXiv:2503.22426.

[140] Yui Oka, Itsumi Saito, Kyosuke Nishida, and Kuniko Saito. Frequency bands in RoPE: Base frequency and context length shape the interpolation–extrapolation trade-off. In *International Conference on Learning Representations (ICLR)*, 2026. No arXiv id found.

[141] Mika Okamoto and Gabriele Sarti. Encoded early, used late: Where transformers begin to act on an inferred partner's expertise. In *Sci-FM Workshop at COLM*, 2026. arXiv:2609.07139.

[142] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.

[143] Nathan Ordonez and Thomas Parnell. MiniPIC: Flexible position-independent caching in <100LOC. *arXiv preprint arXiv:2606.13126*, 2026. 11 Jun 2026, IBM Research. Stores "unrotated K vectors in the KV cache" and "applies RoPE to K tiles inside attention using per-request logical positions" – exactly the position-free storage plus re-rotation-at-a-new-slot that halo recompute needs, implemented in under 100 lines of vLLM.

[144] Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state RNNs. In *Proceedings of EMNLP*, 2024. arXiv:2401.06104.

[145] Mark Oskin. Off-axis, on purpose: Where a transformer computes concepts and why it does so. *arXiv preprint arXiv:2608.10251*, 2026.

[146] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[147] Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Conference on Computational Natural Language Learning (CoNLL)*, 2023. arXiv:2311.04897.

[148] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, 2024. arXiv:2403.12968.

[149] Nilesh Prasad Pandey, Jason Kong, Lanxiang Hu, Quanling Zhao, Yujie Zhao, Onat Gungor, Hao Zhang, and Tajana Rosing. AgentKVShift: Efficient KV cache reuse for agentic memory systems. *arXiv preprint arXiv:2607.21604*, 2026.

[150] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2309.00071.

[151] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2108.12409.

[152] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. arXiv:2210.03350.

[153] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM Web Conference (WWW)*, 2025. arXiv:2409.05591.

[154] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. In *USENIX Conference on File and Storage Technologies (FAST)*, 2025. arXiv:2407.00079.

[155] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2020.

[156] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 2023. arXiv:2302.00083.

[157] Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. Parallel context windows for large language models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2212.10947.

[158] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. SparQ attention: Bandwidth-efficient LLM inference. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2312.04985.

[159] Jonathan Roberts, Kai Han, and Samuel Albanie. Needle threading: Can LLMs follow threads through near-million-scale haystacks? In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.05000.

[160] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. In *International Conference on Machine Learning (ICML)*, 2016. arXiv:1605.06065, under the title "One-shot Learning with Memory-Augmented Neural Networks".

[161] Ning Shang, Li Lyna Zhang, Siyuan Wang, Gaokai Zhang, Gilsinia Lopez, Fan Yang, Weizhu Chen, and Mao Yang. LongRoPE2: Near-lossless LLM context window scaling. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.20082.

[162] Luohe Shi, Zuchao Li, Lefei Zhang, Guoming Liu, Baoyuan Qi, and Hai Zhao. KV-latent: Dimensional-level KV cache reduction with frequency-aware rotary positional embedding. In *Proceedings of ACL*, 2025. arXiv:2507.11273.

[163] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. REPLUG: Retrieval-augmented black-box language models. In *Proceedings of the Conference of the North American Chapter of the ACL (NAACL)*, 2024. arXiv:2301.12652.

[164] Xi Shi, Mengxin Zheng, and Qian Lou. KVShareArena: KV-Cache reuse across contexts and model checkpoints. *arXiv preprint arXiv:2609.10266*, 2026.

[165] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.02542.

[166] Donghyun Son, Euntae Choi, and Sungjoo Yoo. NSNQuant: A double normalization approach for calibration-free low-bit vector quantization of KV cache. *arXiv preprint arXiv:2505.18231*, 2025.

[167] Aryan Sood, Shantanu Acharya, and Gaurav Kumar Nayak. Dropping the anchor: Statistical context summarization for distributed systems via pulsar attention. *arXiv preprint arXiv:2607.20457*, 2026.

[168] Brady Steele. On the limits of learned importance scoring for KV cache compression. *arXiv preprint arXiv:2601.14279*, 2026.

[169] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[170] Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. DRAGIN: Dynamic retrieval augmented generation based on the information needs of large language models. *arXiv preprint arXiv:2403.10081*, 2024.

[171] Zunhai Su and Kehong Yuan. KVSink: Understanding and enhancing the preservation of attention sinks in KV cache quantization for LLMs. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2508.04257.

[172] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.21465.

[173] Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2402.17762.

[174] Simeng Sun, Kalpesh Krishna, Andrew Mattarella-Micke, and Mohit Iyyer. Do long-range language models actually use long-range context? In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021. arXiv:2109.09115.

[175] Fumihiko Tachibana, Daisuke Miyashita, and Jun Deguchi. Fine-tuning a KV cache concatenation-aware model or recomputing KV caches? why not both? *arXiv preprint arXiv:2609.09768*, 2026. 9 Sep 2026. Combines concatenation-aware fine-tuning with selective recomputation of a KV subset; +9.7 RULER points at 124k tokens with 80

[176] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. RazorAttention: Efficient KV cache compression through retrieval heads. *arXiv preprint arXiv:2407.15891*, 2024. commonly cited as ICLR 2025, unverified.

[177] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2406.10774.

[178] Zhenxu Tian, Yi Su, Juntao Li, and Min Zhang. Where matters more than what: Decoding-aligned KV cache compression via position-aware pseudo queries. *arXiv preprint arXiv:2603.11564*, 2026.

[179] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2212.10509.

[180] Szymon Tworkowski, Konrad Staniszewski, Mikołaj Pacek, Yuhuai Wu, Henryk Michalewski, and Piotr Miłoś. Focused transformer: Contrastive training for context scaling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2307.03170.

[181] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. arXiv:1905.09418.

[182] Liang Wang, Nan Yang, and Furu Wei. Query2doc: Query expansion with large language models. In *Proceedings of EMNLP*, 2023. arXiv:2303.07678.

[183] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.07174.

[184] Xi Wang, Taketomo Isazawa, Liana Mikaelyan, and James Hensman. KBLaM: Knowledge base augmented language model. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10450.

[185] Xinghao Wang, Pengyu Wang, Xiaoran Liu, Fangxu Liu, Jason Chu, Kai Song, and Xipeng Qiu. Prism: Spectral-aware block-sparse attention. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.08426.

[186] Yifei Wang, Yueqi Wang, Zhenrui Yue, Huimin Zeng, Yong Wang, Ismini Lourentzou, Zhengzhong Tu, Xiangxiang Chu, and Julian McAuley. FASA: Frequency-aware sparse attention. In *ICLR*, 2026. arXiv:2602.03152.

[187] Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. Lookahead q-cache: Achieving more consistent KV cache eviction via pseudo query. In *Proceedings of EMNLP*, 2025. arXiv:2505.20334.

[188] Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MEMORYLLM: Towards self-updatable large language models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.04624.

[189] Jeffrey Willette, Heejun Lee, Youngwan Lee, Myeongjae Jeon, and Sung Ju Hwang. Training-free exponential context extension via cascading KV cache. *arXiv preprint arXiv:2406.17808*, 2024.

[190] Wei Wu, Zhuoshi Pan, Chao Wang, Liyi Chen, Yunchu Bai, Tianfu Wang, Kun Fu, Zheng Wang, and Hui Xiong. TokenSelect: Efficient long-context inference and length extrapolation for LLMs via dynamic token-level KV cache selection. In *Proceedings of EMNLP*, 2025. arXiv:2411.02886.

[191] Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality. *arXiv preprint arXiv:2404.15574*, 2024.

[192] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2203.08913.

[193] Haocheng Xia, Mihir Pamnani, Hanxi Fang, Supawit Chockchowwat, and Yongjoo Park. LazyAttention: Efficient retrieval-augmented generation with deferred positional encoding. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.04302.

[194] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.04617.

[195] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[196] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2025.

[197] Jihao Xin, Tian Lyu, David Keyes, Hatem Ltaief, and Marco Canini. RAP: KV-cache compression via RoPE-aligned pruning. *arXiv preprint arXiv:2602.02599*, 2026.

[198] Chejian Xu, Wei Ping, Peng Xu, Zihan Liu, Boxin Wang, Mohammad Shoeybi, Bo Li, and Bryan Catanzaro. From 128k to 4m: Efficient training of ultra-long context large language models. *arXiv preprint arXiv:2504.06214*, 2025.

[199] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.04408.

[200] Yuhui Xu, Zhanming Jie, Hanze Dong, Lei Wang, Xudong Lu, Aojun Zhou, Amrita Saha, Caiming Xiong, and Doyen Sahoo. ThinK: Thinner key cache by query-driven pruning. In *ICLR*, 2025. arXiv:2407.21018.

[201] Hongkang Yang, Zehao Lin, Wenjin Wang, Hao Wu, Zhiyu Li, Bo Tang, Wenqiang Wei, Jinbo Wang, Zeyun Tang, Shichao Song, Chenyang Xi, Yu Yu, Kai Chen, Feiyu Xiong, Linpeng Tang, and Weinan E. Memory^3: Language modeling with explicit memory. *Journal of Machine Learning*, 3:300–346, 2024. arXiv:2407.01178.

[202] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large language models via efficient KV cache reuse. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.16002.

[203] Lijie Yang, Zhihao Zhang, Zhuofu Chen, Zikun Li, and Zhihao Jia. TidalDecode: Fast and accurate LLM decoding with position persistent sparse attention. *arXiv preprint arXiv:2410.05076*, 2024.

[204] Xintong Yang, Hao Gu, Binxing Xu, Lujun Li, Bei Liu, Jiacheng Liu, Qiyuan Zhu, Yike Guo, and Sirui Han. IndexMem: Learned KV-cache eviction with latent memory for long-context LLM inference. *arXiv preprint arXiv:2605.25475*, 2026.

[205] Xinyu Yang, Tianqi Chen, and Beidi Chen. APE: Faster and longer context-augmented generation via adaptive parallel encoding. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.05431.

[206] Zi Yang and Nan Hua. Attendre: Wait to attend by retrieval with evicted queries in memory-based transformers for long context processing. *arXiv preprint arXiv:2401.04881*, 2024.

[207] Hengshuai Yao, Xing Chen, Ahmed Murtadha, and Guan Wang. Thin keys, full values: Reducing KV cache via low-dimensional attention selection. *arXiv preprint arXiv:2603.04427*, 2026.

[208] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2405.16444.

[209] Yao Yao, Zuchao Li, and Hai Zhao. SirLLM: Streaming infinite retentive LLM. *arXiv preprint arXiv:2405.12528*, 2024.

[210] Yiming Yao, Chenyang Lyu, Xuanfan Ni, Longyue Wang, Weihua Luo, Yazheng Yang, and Jinsong Su. WnW: Waxing-and-waning KV cache for long-form speech LLMs. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026. arXiv:2608.22704.

[211] Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Weichuan Liu, Lei Hou, and Juanzi Li. SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation. *arXiv preprint arXiv:2406.19215*, 2024.

[212] Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02694.

[213] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

[214] Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, Guohao Dai, Shengen Yan, and Yu Wang. LV-Eval: A balanced long-context benchmark with 5 length levels up to 256k. *arXiv preprint arXiv:2402.05136*, 2024.

[215] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. PQCache: Product quantization-based KVCache for long context LLM inference. *Proceedings of the ACM on Management of Data (PACMMOD)*, 2025. arXiv:2407.12820.

[216] Haoyue Zhang, Hualei Zhang, Xiaosong Ma, Jie Zhang, and Song Guo. LazyEviction: Lagged KV eviction with attention pattern observation for efficient long reasoning. *arXiv preprint arXiv:2506.15969*, 2025.

[217] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*, 2024.

[218] Wuwei Zhang, Fangcong Yin, Howard Yen, Danqi Chen, and Xi Ye. Query-focused retrieval heads improve long-context reasoning and re-ranking. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2506.09944.

[219] Wuyang Zhang and Shichao Pei. Predictive prefetching for retrieval-augmented generation. In *Proceedings of ICML*, 2026. arXiv:2605.17989.

[220] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. ∞bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.13718.

[221] Yike Zhang, Zhiyuan He, Huiqiang Jiang, Chengruidong Zhang, Yuqing Yang, Jianyong Wang, and Lili Qiu. LeanK: Learnable K cache channel pruning for efficient decoding. In *Proceedings of EMNLP*, 2025. arXiv:2508.02215.

[222] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023.

[223] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.

[224] Zifan Zheng, Yezhaohui Wang, Yuxin Huang, Shichao Song, Mingchuan Yang, Bo Tang, Feiyu Xiong, and Zhiyu Li. Attention heads of large language models: A survey. *arXiv preprint arXiv:2409.03752*, 2024.

[225] Yuhao Zhou, Sirui Song, Boyang Liu, Zhiheng Xi, Senjie Jin, Xiaoran Fan, Zhihao Zhang, Wei Li, and Xuanjing Huang. EliteKV: Scalable KV cache compression via RoPE frequency selection and joint low-rank projection. *arXiv preprint arXiv:2503.01586*, 2025.